

ID3 Algorithm

Implementing an ID3 Decision Tree Classifier in C++
and Deploying it to FPGA-Targeted Hardware

*A complete engineering account of the design, verification,
optimisation, and synthesis of a binary decision tree classifier*

Abstract

This paper documents the end-to-end design and implementation of a decision tree classifier, beginning with the information-theoretic foundations of the ID3 algorithm and concluding with a synthesised, hardware-verified digital circuit targeted at Xilinx FPGA architecture.

The work proceeds in five phases. First, a decision tree learner was implemented from scratch in C++, including entropy calculation, information gain evaluation, recursive tree construction, pre-pruning stopping conditions, and prediction. Second, the resulting tree was *flattened* — converted from a recursive, index-linked structure into a fixed-size table suitable for hardware consumption — and exported as a SystemVerilog header with automatically computed minimal bit-widths. Third, a combinational SystemVerilog module was written to walk this flattened tree, verified against the C++ reference using a fourteen-vector self-checking testbench. Fourth, two optimisation studies were conducted: a bit-width reduction study and a combinational-versus-pipelined architectural comparison. Fifth, the design was synthesised against Xilinx 7-series primitives and the resulting gate-level netlist re-verified for functional correctness.

Key quantitative results: bit-width minimisation reduced synthesised cell count by **43%** (174 → 99 cells); pipelining the tree walk to one level per clock cycle reduced cell count by a further **53%** (99 → 46 cells) at a cost of three clock cycles of latency per inference; and Xilinx-targeted synthesis mapped the entire combinational classifier to **a single LUT4** plus I/O buffers, with all fourteen test vectors passing on the post-synthesis netlist.

Beyond the numerical results, this paper deliberately documents the *process* — including defects encountered, misconceptions corrected, tooling failures overcome, and design decisions reconsidered — on the basis that an honest account of engineering practice is more instructive than a sanitised presentation of final results.

Table 1: Summary of headline results.

Optimisation	Before	After	Change
Bit-width minimisation	174 cells	99 cells	−43%
Combinational → pipelined	99 cells	46 cells	−53%
Xilinx LUT mapping	—	1 LUT4	—
Functional verification	14/14 vectors pass at RTL and post-synthesis		

Contents

1	Introduction	4
1.1	Motivation	4
1.2	Project Scope	4
1.3	Toolchain Summary	4
2	Theoretical Background	5
2.1	What a Decision Tree Is	5
2.2	The Play Tennis Dataset	5
2.3	Feature Encoding	6
2.4	Entropy	6
2.5	Information Gain	7
2.6	The ID3 Algorithm	8
2.7	Binary Splits versus Multi-way Splits	8
2.8	Overfitting and Pruning	9
3	Software Implementation in C++	10
3.1	Data Representation	10
3.2	entropy() — Measuring Impurity	11
3.3	information_gain() — Scoring a Candidate Split	11
3.4	majority_class() — The Fallback Prediction	12
3.5	build_tree() — Recursive Construction	13
3.6	predict() — Inference	15
3.7	Two Defects and Their Resolution	16
3.7.1	Defect 1: Undefined behaviour on an empty subset	16
3.7.2	Defect 2: Non-terminating recursion	16
3.8	Pre-Pruning: Three Stopping Conditions	17
3.9	Validation Methodology	18
3.10	Hyperparameter Sweep	19
3.11	Which Features the Tree Actually Uses	19
4	Flattening: Bridging Software and Hardware	20
4.1	Why Flattening Is Necessary	20
4.2	Designing the Encoding	20
4.3	flatten_tree() — Producing the Table	21
4.4	The Index-Remapping Defect	23
4.5	export_vh() — Emitting the Hardware Header	24
4.6	Bit-Width Analysis	26
5	The Toolchain	26
5.1	Why Ubuntu, and What WSL Is	26
5.2	Icarus Verilog — Simulation	27
5.3	GTKWave — Waveform Visualisation	27
5.4	Yosys — Synthesis	30
5.5	Command Reference	30
6	Hardware Implementation in SystemVerilog	31
6.1	The Combinational Module	31
6.1.1	Module Declaration and Ports	32
6.1.2	always_comb — Combinational Behaviour	32
6.1.3	The Loop as a Recursion Substitute	33
6.1.4	The done Flag	33

6.1.5	Bit-Slicing	33
6.1.6	Correspondence to <code>predict()</code>	34
6.2	The Self-Checking Testbench	34
7	Optimisation Studies	35
7.1	Overview of Strategies	35
7.2	Study 1 — Bit-Width Minimisation	36
7.3	Pipelining — Theory	37
7.4	Pipelined Implementation	37
7.5	The Clocked Testbench	38
7.6	Architectural Comparison	39
8	Synthesis and Post-Synthesis Verification	40
8.1	Xilinx-Targeted Synthesis	40
8.2	Post-Synthesis Netlist Verification	41
9	Engineering Challenges Encountered	42
9.1	Truncated File Producing Misattributed Errors	42
9.2	Pipeline–Exporter Divergence	43
10	Conceptual Clarifications	43
11	Limitations	44
11.1	On the Parameterisation Gap	44
12	Future Work	45
12.1	Deployment Path	45
13	Conclusion	45
A	The Final Tree	46
B	Complete Predictions	47
C	Reproduction Instructions	47
D	Repository Contents	48
E	Glossary	48

1 Introduction

1.1 Motivation

Machine learning inference is increasingly deployed at the edge — in embedded systems, sensors, and real-time control loops where a general-purpose processor is either unavailable, too slow, or too power-hungry. Decision trees are a particularly attractive candidate for hardware implementation: unlike neural networks, they require no multiplication, no floating-point arithmetic, and no large weight matrices. A trained decision tree reduces to a sequence of comparisons and branches — precisely the kind of operation digital logic performs natively and cheaply.

This project asks a concrete question: **what does it actually take to move a decision tree from a C++ program into a digital circuit?** The answer turns out to involve considerably more than a mechanical translation. Software data structures rely on dynamic memory, pointers, and recursion — none of which exist in hardware. Bridging that gap requires rethinking how the tree is represented, how it is traversed, and how much physical circuitry each design choice costs.

1.2 Project Scope

Table 2: Project deliverables by phase.

Phase	Deliverable	Language / Tool
1	Decision tree learner (training + inference)	C++17
2	Tree flattening and hardware header export	C++17
3	Combinational tree-walk module	SystemVerilog
4	Self-checking testbench (14 vectors)	SystemVerilog
5	Pipelined (clocked) tree-walk module	SystemVerilog
6	Bit-width and architecture optimisation studies	Yosys
7	FPGA-targeted synthesis and netlist verification	Yosys + Icarus

Explicitly **out of scope**: physical deployment to an FPGA board. No hardware was available at the time of writing; the project therefore terminates at synthesis, which is a complete and standard verification milestone in its own right.

1.3 Toolchain Summary

Table 3: Tools used, with justification for each selection.

Tool	Purpose	Why chosen
g++ (C++17)	Compiling the tree learner	Standard, available, project already in C++
WSL2 + Ubuntu	Linux environment on a Windows host	Hardware toolchains are Linux-native; package installation is trivial
Icarus Verilog 12.0	SystemVerilog simulation	Free, lightweight, supports <code>always_comb/always_ff</code>
GTKWave 3.3	Waveform visualisation	Standard companion to Icarus; visual debugging
Yosys 0.33	Logic synthesis and resource analysis	Free; provides both generic and Xilinx-targeted synthesis

2 Theoretical Background

2.1 What a Decision Tree Is

A decision tree is a supervised learning model that classifies examples by asking a sequence of questions about their features. Structurally it consists of:

- **Root node** — the entry point, containing the entire training set
- **Internal (decision) nodes** — each asks a question about one feature (e.g. “*is Humidity less than 1?*”)
- **Branches** — the outcomes of a question, leading to child nodes
- **Leaf nodes** — terminal nodes containing a final prediction

Classification proceeds by starting at the root, evaluating that node’s question against the example being classified, following the appropriate branch, and repeating until a leaf is reached. The leaf’s stored value is the prediction.

The learning problem is therefore: **given a training set, which questions should be asked, in which order?**

2.2 The Play Tennis Dataset

The dataset used throughout is the classic *Play Tennis* dataset, originally popularised by Quinlan and used as the worked example in Tom Mitchell’s *Machine Learning*. It contains fourteen days of weather observations and a binary label indicating whether tennis was played.

Table 4: The Play Tennis dataset in its original categorical form.

Day	Outlook	Temperature	Humidity	Wind	Play Tennis
D1	Sunny	Hot	High	Weak	No
D2	Sunny	Hot	High	Strong	No
D3	Overcast	Hot	High	Weak	Yes
D4	Rain	Mild	High	Weak	Yes
D5	Rain	Cool	Normal	Weak	Yes
D6	Rain	Cool	Normal	Strong	No
D7	Overcast	Cool	Normal	Strong	Yes
D8	Sunny	Mild	High	Weak	No
D9	Sunny	Cool	Normal	Weak	Yes
D10	Rain	Mild	Normal	Weak	Yes
D11	Sunny	Mild	Normal	Strong	Yes
D12	Overcast	Mild	High	Strong	Yes
D13	Overcast	Hot	Normal	Weak	Yes
D14	Rain	Mild	High	Strong	No

Why this dataset was chosen. Three properties make it ideal:

1. **It is small enough to compute by hand.** Every entropy and information gain figure in this paper was verified manually before being trusted from code — a discipline that caught several errors.
2. **All features are categorical**, avoiding the complication of continuous-valued thresholds during learning.
3. **It is a canonical benchmark**, so results can be sanity-checked against published values.

Class balance. The target contains 9 “Yes” and 5 “No” — imbalanced enough to make the entropy calculation non-trivial, but not so skewed that a majority-class baseline would be uninformative.

2.3 Feature Encoding

Because the eventual hardware implementation operates on integers, all categorical values were encoded numerically.

Table 5: Numeric encoding of categorical features.

Feature	Index	Encoding
Outlook	0	Sunny = 0, Overcast = 1, Rain = 2
Temperature	1	Hot = 0, Mild = 1, Cool = 2
Humidity	2	High = 0, Normal = 1
Wind	3	Weak = 0, Strong = 1
Play (label)	—	No = 0, Yes = 1

```

1 std::vector<std::vector<float>> X = {
2     {0,0,0,0}, {0,0,0,1}, {1,0,0,0}, {2,1,0,0}, {2,2,1,0},
3     {2,2,1,1}, {1,2,1,1}, {0,1,0,0}, {0,2,1,0}, {2,1,1,0},
4     {0,1,1,1}, {1,1,0,1}, {1,0,1,0}, {2,1,0,1}
5 };
6 std::vector<int> y = {0,0,1,1,1,0,1,0,1,1,1,1,1,0};

```

Listing 1: The encoded dataset as it appears in the C++ source.

Each row of X is one day, with the four values in the fixed order {Outlook, Temperature, Humidity, Wind}. The corresponding entry in y is that day’s label.

A note on this encoding. Assigning integers to unordered categories introduces an implicit ordering that does not exist in the real world — encoding Rain as 2 does not make it “greater than” Sunny in any meaningful sense. This becomes significant in Section 2.7, where it interacts with the choice of binary versus multi-way splitting. It is a deliberate simplification, made because the hardware target requires numeric comparison, and its consequences are analysed rather than ignored.

2.4 Entropy

Entropy quantifies the *impurity* or *disorder* of a set of labels. For a set S with classes indexed by i :

$$H(S) = - \sum_i p_i \log_2(p_i) \quad (1)$$

where p_i is the proportion of examples in S belonging to class i .

Interpreting the extremes.

- If every example shares the same label, one $p_i = 1$ and the rest are 0. Since $\log_2(1) = 0$, entropy is **0** — a perfectly pure set.
- If two classes are evenly split, $p_1 = p_2 = 0.5$, giving $H = -(0.5 \log_2 0.5 + 0.5 \log_2 0.5) = 1$ — maximum impurity for a binary problem.

Worked calculation for the full dataset. With 9 “Yes” and 5 “No” out of 14:

$$p_{\text{yes}} = \frac{9}{14} = 0.643, \quad p_{\text{no}} = \frac{5}{14} = 0.357 \quad (2)$$

$$\log_2(0.643) = \frac{\ln(0.643)}{\ln(2)} = \frac{-0.442}{0.693} = -0.638 \quad (3)$$

$$\log_2(0.357) = \frac{\ln(0.357)}{\ln(2)} = \frac{-1.030}{0.693} = -1.487 \quad (4)$$

$$\begin{aligned} H(S) &= -(0.643)(-0.638) - (0.357)(-1.487) \\ &= 0.410 + 0.531 = \mathbf{0.941} \end{aligned} \quad (5)$$

The implementation returns `0.940286`, agreeing with the hand calculation to within rounding error, and with the value published in the standard literature.

A subtlety worth recording. The convention $0 \log_2 0 = 0$ is required for the formula to be well-defined when a class is absent. Mathematically this is justified as a limit: $\lim_{p \rightarrow 0} p \log_2 p = 0$. In code this is handled by an explicit `if (p > 0)` guard, because $\log_2(0)$ evaluates to negative infinity and would otherwise poison the entire sum.

2.5 Information Gain

Information gain measures how much a proposed split *reduces* entropy. For a set S split on attribute A :

$$\text{Gain}(S, A) = H(S) - \sum_{v \in \text{values}(A)} \frac{|S_v|}{|S|} H(S_v) \quad (6)$$

The subtracted term is the **weighted average entropy of the children**, where each child's entropy is weighted by the fraction of examples it received. The weighting matters: a split producing one tiny pure group and one large impure group should not score as well as one producing two moderately pure groups of equal size.

$$\begin{aligned} \text{Entropy}(S) &= \sum_{i=1}^c -p_i \log_2 p_i \\ \text{Gain}(S, A) &= \text{Entropy}(S) - \sum_{v \in \text{Values}(A)} \frac{|S_v|}{|S|} \text{Entropy}(S_v) \end{aligned}$$

Figure 1: The information gain formula as originally worked through by hand during the theoretical phase of the project. The summation runs over every value v of attribute A , with each child's entropy weighted by $|S_v|/|S|$.

Worked calculation — splitting on Humidity. Humidity partitions the fourteen examples into High (7 examples: 3 Yes, 4 No) and Normal (7 examples: 6 Yes, 1 No).

$$H(\text{High}) = -\left(\frac{3}{7} \log_2 \frac{3}{7} + \frac{4}{7} \log_2 \frac{4}{7}\right) = 0.985 \quad (7)$$

$$H(\text{Normal}) = -\left(\frac{6}{7} \log_2 \frac{6}{7} + \frac{1}{7} \log_2 \frac{1}{7}\right) = 0.592 \quad (8)$$

$$\text{Weighted} = \frac{7}{14}(0.985) + \frac{7}{14}(0.592) = 0.789 \quad (9)$$

$$\text{Gain}(\text{Humidity}) = 0.940 - 0.789 = \mathbf{0.151} \quad (10)$$

The implementation returns 0.151836.

Table 6: Information gain for all four features under classical multi-way splitting, computed by hand.

Feature	Weighted child entropy	Information gain
Outlook	0.694	0.246
Humidity	0.789	0.151
Wind	0.892	0.048
Temperature	0.911	0.029

Under classical ID3, **Outlook** would therefore be selected as the root — the standard textbook result. The Overcast branch is notable: all four Overcast days are “Yes”, giving that subset an entropy of exactly zero and allowing it to terminate immediately as a pure leaf.

2.6 The ID3 Algorithm

The ID3 procedure is recursive:

```

ID3(Examples, Attributes):
  if all Examples share one label:
    return a leaf with that label
  if Attributes is empty:
    return a leaf with the majority label
  A <- the attribute with highest Gain(Examples, A)
  for each value v of A:
    Examples_v <- the subset of Examples with A = v
    if Examples_v is empty:
      attach a leaf with the parent's majority label
    else:
      attach ID3(Examples_v, Attributes - A)
  return the resulting node

```

Listing 2: ID3 in pseudocode.

Three base cases are required, not one. This is worth emphasising, because the initial implementation in this project included only the first and consequently contained two latent defects (Section 3.7).

- **All examples share a label** — the node is pure; no further splitting can help.
- **No attributes remain** — the node is impure but nothing is left to split on; return the majority.
- **A branch receives no examples** — nothing exists to recurse on; fall back to the parent's majority.

Note also $\text{Attributes} - A$ in the recursive call. Once a categorical attribute has been split on, every example in that branch shares the same value for it, so the attribute conveys zero remaining information within that branch. Removing it guarantees the attribute list strictly shrinks on every recursive call, which in turn guarantees termination.

2.7 Binary Splits versus Multi-way Splits

Classical ID3 performs a **multi-way split**: an attribute with three values produces three branches simultaneously. The implementation in this project instead performs **binary splits** using a numeric threshold ($\text{feature} < \text{threshold}$), closer to the CART family of algorithms.

This is a deliberate design decision with real consequences, and it was measured rather than assumed.

Table 7: Label distribution by Outlook value. Overcast is perfectly pure.

Outlook value	Count	Labels
Sunny (0)	5	2 Yes, 3 No
Overcast (1)	4	4 Yes, 0 No — perfectly pure
Rain (2)	5	3 Yes, 2 No

A multi-way split isolates the Overcast group cleanly, contributing zero entropy. A binary split *cannot* — it must lump Overcast in with either Sunny or Rain, contaminating an otherwise pure subset. This is not a limitation of the gain metric; it is a structural limitation of what a binary threshold can express.

Table 8: Measured impact of split type on Outlook, training subset.

Split type on Outlook	Information gain
Best binary threshold	0.105
Full 3-way split	0.332

The multi-way split scores approximately three times higher. This directly explains why the implemented tree selects **Humidity** as its root rather than Outlook: Humidity is naturally binary, so its single available binary split *is* its full split, and its gain (0.152) exceeds Outlook’s best *binary* option (0.105) — even though Outlook’s *multi-way* gain (0.246) exceeds Humidity’s.

A controlled experiment. To test whether this mattered in practice, a multi-way variant was implemented and compared on identical data with an identical holdout split.

Table 9: Binary versus multi-way splitting, controlled comparison.

Implementation	Nodes	Holdout accuracy
Binary splits	11	2/3
Multi-way splits	8	2/3

The multi-way tree was structurally more compact and scored substantially higher gains during training, **yet produced identical holdout accuracy**. This is a genuinely instructive negative result: better separation of *training* data does not automatically translate into better generalisation, particularly when the evaluation set contains only three examples. The finding is reported here rather than suppressed.

Why binary splitting was retained. Beyond the empirical tie, binary splitting has a decisive advantage for the hardware target: every node has exactly two children, so the flattened representation has a *fixed* row width. Multi-way splitting produces nodes with a variable number of children, requiring either padding (wasting memory on nodes with fewer branches) or a variable-length encoding — significantly complicating the hardware design for no measured accuracy benefit on this dataset.

2.8 Overfitting and Pruning

Left unconstrained, a decision tree will subdivide the feature space until every leaf is pure — memorising the training set, including its noise. Such a tree achieves perfect training accuracy and generalises poorly.

Table 10: Comparison of pruning strategies.

	Pre-pruning	Post-pruning
Mechanism	Halt growth during construction	Grow fully, then remove unhelpful branches
Cost	Cheap — avoids building the full tree	Expensive — full tree built first
Risk	Short-sighted; may stop before a useful deeper split	None structurally; more thorough
Typical use	Quick baselines, large datasets	When the best-tuned tree is required
Implemented here	Yes (Section 3.8)	No (see Future Work)

Cost-complexity pruning is the standard post-pruning formulation, minimising

$$R_\alpha(T) = \text{Error}(T) + \alpha \cdot |\text{leaves}(T)| \quad (11)$$

over a range of α , selecting the value that maximises cross-validated accuracy. This project implements pre-pruning only; post-pruning is discussed as future work.

3 Software Implementation in C++

3.1 Data Representation

The tree is represented not as a network of heap-allocated nodes linked by pointers, but as a **flat vector of nodes addressed by integer index**.

```

1 struct Node {
2     int    feature_index;      // which column of X this node tests
3     float  threshold;         // the cutoff value it compares against
4     int    left_child_index;   // index into 'tree' of the "true" child
5     int    right_child_index; // index into 'tree' of the "false" child
6     int    leaf_value;        // the prediction, if this is a leaf
7     bool   is_leaf;           // whether this node is terminal
8 };
9
10 std::vector<Node> tree;
```

Listing 3: The Node structure and the global tree storage.

This choice was made at the outset and proved consequential.

1. **No manual memory management.** There is no `new`, no `delete`, and consequently no possibility of a memory leak or double-free — an entire class of defect eliminated by construction.
2. **Trivially serialisable.** Because nodes already reference each other by array position rather than by memory address, exporting the tree requires no pointer-chasing or address translation.
3. **Structurally aligned with the hardware target.** Digital logic cannot dereference pointers. A design that already addresses children by index is far closer to hardware-ready than one built on pointers.

A comparison implementation encountered during background research used a linked structure (`TreeNode*` with an `EdgeNode*` child list) requiring an explicit recursive `deallocate_tree` function. That approach supports variable-arity nodes, but carries manual memory management risk and is substantially harder to flatten.

3.2 entropy() — Measuring Impurity

```
1 float entropy(std::vector<int> labels){
2     std::map<int, int> counts;
3     for (int label : labels){
4         counts[label]++;
5     }
6     float total = labels.size();
7     float result = 0.0;
8     for (std::map<int, int>::iterator it = counts.begin();
9          it != counts.end(); it++) {
10         int count = it->second;
11         float p = count / total;
12         if (p > 0) {
13             result = result - (p * log2(p));
14         }
15     }
16     return result;
17 }
```

Listing 4: Entropy calculation.

Operation. The function proceeds in two passes. The first builds a frequency table: `std::map<int,int> counts` maps each distinct label to its occurrence count, and `counts[label]++` either creates a value-initialised entry or increments an existing one. For input `[1,1,0,1,0]` this yields `{0: 2, 1: 3}`. The second pass iterates over the *distinct* labels — not the original examples — computing each proportion p and accumulating $-p \log_2 p$.

Design details worth noting.

- `float total = labels.size()` performs the cast to floating point *before* the division. Had `total` remained an integer, `count / total` would perform integer division, silently truncating every proportion to zero and returning an entropy of zero for all inputs — a subtle and severe defect.
- The `if (p > 0)` guard implements the $0 \log_2 0 = 0$ convention from Section 2.4. In this function the guard can never fire, since the map contains only labels that occurred; it is retained as defensive practice.
- The function is **class-count agnostic**. Nothing restricts it to binary labels — a three-class problem would work unmodified.

Behaviour on empty input. Given an empty vector, both loops execute zero times and the function returns its initialiser, 0.0. This is mathematically defensible but proved to be the root cause of a serious defect (Section 3.7).

3.3 information_gain() — Scoring a Candidate Split

```
1 float information_gain(std::vector<std::vector<float>> X, std::vector<int> y,
2                       int feature_index, float threshold){
3     std::vector<int> left_labels;
4     std::vector<int> right_labels;
5     for(int i = 0; i < X.size(); i++){
6         if(X[i][feature_index] < threshold){
7             left_labels.push_back(y[i]);
8         } else {
9             right_labels.push_back(y[i]);
10        }
11    }
```

```

12     float ent_before = entropy(y);
13
14     float n_left  = left_labels.size();
15     float n_right = right_labels.size();
16     float n_total = y.size();
17
18     float weighted_entropy_avg = (n_left / n_total) * entropy(left_labels)
19                                   + (n_right / n_total) * entropy(right_labels);
20
21     return ent_before - weighted_entropy_avg;
22 }

```

Listing 5: Information gain for one candidate split.

Operation. The function partitions the labels according to whether each example’s value for `feature_index` falls below `threshold`, then applies Equation 6 directly. Only the *labels* are partitioned, not the full feature rows — the feature values are not needed to compute entropy, so copying them would be wasted work.

Correspondence to theory. The variable `weighted_entropy_avg` implements $\sum_v \frac{|S_v|}{|S|} H(S_v)$ exactly, with two terms corresponding to the left and right partitions. Because splits are binary, the summation always has precisely two terms.

A comparison worth noting. A reference implementation encountered during research computed its split criterion as

```

1 tmp = sub_groups_out[i].size();
2 entropy = entropy + tmp * std::log2(tmp);

```

which depends only on subgroup *sizes* and never examines the label distribution within each subgroup. Such a metric cannot distinguish a split producing two clean, well-separated groups from one producing two thoroughly mixed groups of the same sizes. The implementation presented here uses the correct entropy-weighted formulation, verified against hand calculations.

3.4 majority_class() — The Fallback Prediction

```

1 int majority_class(std::vector<int> y){
2     std::map<int, int> counts;
3     for (int label : y) {
4         counts[label]++;
5     }
6
7     int best_label = -1;
8     int best_count = -1;
9
10    for (const auto& [label, count] : counts) {
11        if (count > best_count) {
12            best_count = count;
13            best_label = label;
14        }
15    }
16    return best_label;
17 }

```

Listing 6: Majority class selection.

Operation. An identical counting pass to `entropy()`, followed by a linear scan for the maximum — the classic “track the best seen so far” pattern. Sentinel values of `-1` are safe because genuine counts are always at least 1. The loop uses C++17 **structured bindings** (`const auto& [label, count]`), unpacking each map entry directly into two named variables — equivalent to the older iterator form used in `entropy()`, but considerably more readable.

Purpose. A leaf must emit a single prediction, but the data reaching a leaf is not always pure. When a stopping condition forces termination on a mixed node, `majority_class` supplies the most defensible available answer: the label that would be correct most often, given what was observed.

An observation on code duplication. The counting pass here is byte-for-byte identical to the one in `entropy`. Extracting a shared `count_labels()` helper would remove the duplication and localise any future change. The duplication is documented rather than concealed.

3.5 `build_tree()` — Recursive Construction

```
1 Node build_tree(std::vector<std::vector<float>> X, std::vector<int> y,
2               int max_depth = 2, int current_depth = 0,
3               int min_samples_split = 5, float purity_threshold = 0.5){
4     Node leaf_node;
5
6     // --- Stopping condition 1: depth limit ---
7     if(current_depth >= max_depth){
8         leaf_node.is_leaf = true;
9         leaf_node.leaf_value = majority_class(y);
10        return leaf_node;
11    }
12
13    // --- Stopping condition 2: too few samples ---
14    if((int)y.size() < min_samples_split){
15        leaf_node.is_leaf = true;
16        leaf_node.leaf_value = majority_class(y);
17        return leaf_node;
18    }
19
20    // --- Stopping condition 3: sufficiently pure ---
21    if(entropy(y) <= purity_threshold){
22        leaf_node.is_leaf = true;
23        leaf_node.leaf_value = majority_class(y);
24        return leaf_node;
25    }
26
27    // --- Exhaustive search for the best split ---
28    float best_gain = -1;
29    int best_feature = -1;
30    float best_threshold = -1;
31
32    for (int f = 0; f < X[0].size(); f++) {
33        for (int i = 0; i < X.size(); i++) {
34            float candidate_threshold = X[i][f];
35            float gain = information_gain(X, y, f, candidate_threshold);
36            if (gain > best_gain) {
37                best_gain = gain;
38                best_feature = f;
39                best_threshold = candidate_threshold;
40            }
41        }
42    }
```

```

43
44 // --- Stopping condition 4: no split helps ---
45 if (best_gain <= 0) {
46     leaf_node.is_leaf = true;
47     leaf_node.leaf_value = majority_class(y);
48     return leaf_node;
49 }
50
51 // --- Partition the data ---
52 std::vector<std::vector<float>> left_X, right_X;
53 std::vector<int> left_y, right_y;
54 for (int i = 0; i < X.size(); i++) {
55     if (X[i][best_feature] < best_threshold) {
56         left_X.push_back(X[i]);
57         left_y.push_back(y[i]);
58     } else {
59         right_X.push_back(X[i]);
60         right_y.push_back(y[i]);
61     }
62 }
63
64 leaf_node.is_leaf = false;
65 leaf_node.feature_index = best_feature;
66 leaf_node.threshold = best_threshold;
67
68 // --- Recurse, with empty-subset guard ---
69 if (left_y.empty()) {
70     Node left_leaf;
71     left_leaf.is_leaf = true;
72     left_leaf.leaf_value = majority_class(y); // parent's majority
73     tree.push_back(left_leaf);
74     leaf_node.left_child_index = tree.size() - 1;
75 } else {
76     Node left_node = build_tree(left_X, left_y, max_depth,
77                                current_depth + 1, min_samples_split,
78                                purity_threshold);
79     tree.push_back(left_node);
80     leaf_node.left_child_index = tree.size() - 1;
81 }
82
83 if (right_y.empty()) {
84     Node right_leaf;
85     right_leaf.is_leaf = true;
86     right_leaf.leaf_value = majority_class(y);
87     tree.push_back(right_leaf);
88     leaf_node.right_child_index = tree.size() - 1;
89 } else {
90     Node right_node = build_tree(right_X, right_y, max_depth,
91                                current_depth + 1, min_samples_split,
92                                purity_threshold);
93     tree.push_back(right_node);
94     leaf_node.right_child_index = tree.size() - 1;
95 }
96
97 return leaf_node;
98 }

```

Listing 7: The complete tree construction function, with all four stopping conditions and both defect fixes.

Structure. The function has five phases, in strict order:

1. **Stopping conditions** — checked *before* any expensive work. This ordering matters:

placing them after the split search would waste the entire search whenever a condition fires, and would require undoing partially-assigned node fields.

2. **Exhaustive split search** — every feature, every observed value as a candidate threshold.
3. **Gain validity check** — reject the split if it achieved nothing.
4. **Partitioning** — divide the data into two subsets.
5. **Recursion** — build each subtree, with a guard against empty subsets.

The split search. The nested loop tests $|\text{features}| \times |\text{examples}|$ candidates, using every observed feature value as a candidate threshold. This is valid if slightly naive; a more efficient implementation would sort each feature’s values and test only the midpoints between consecutive distinct values, since any threshold between two adjacent values produces an identical partition.

The comparison `if (gain > best_gain)` is the same running-maximum pattern as `majority_class`, extended to track three correlated values. **Note the comparison is strict.** Ties are therefore broken by search order — the first candidate found wins. This is not a principled rule, and it has an observable consequence documented in Section 3.11.

Index recording. The pattern

```
1 tree.push_back(left_node);
2 leaf_node.left_child_index = tree.size() - 1;
```

appears four times. It records a child’s position *immediately after* insertion: `tree.size() - 1` is by definition the index of the element just appended. This is what makes the index-based tree self-consistent without any pointer arithmetic.

3.6 predict() — Inference

```
1 int predict(const Node& node, const std::vector<float>& row){
2     if (node.is_leaf) return node.leaf_value;
3     if (row[node.feature_index] < node.threshold)
4         return predict(tree[node.left_child_index], row);
5     else
6         return predict(tree[node.right_child_index], row);
7 }
```

Listing 8: Inference for a single example.

Eight lines, and the conceptual heart of the whole project — the SystemVerilog module in Section 6 is a direct hardware transliteration.

Operation.

- **Base case:** if the node is a leaf, return its stored prediction.
- **Recursive case:** consult `node.feature_index` to determine *which* feature this node cares about, extract that value with `row[node.feature_index]`, compare against `node.threshold`, and recurse into the corresponding child.

Critically, `predict` makes **exactly one recursive call per node** — it follows a single data-determined path from root to leaf. This distinguishes it from `print_tree` and `flatten_tree`, which visit *both* children unconditionally.

Table 11: Worked trace of `predict()` for {Sunny, Mild, High, Weak} on the final tree.

Step	Node	Feature	Value	Test	Result
1	Root	Humidity (2)	0	$0 < 1?$	True → left
2	Outlook node	Outlook (0)	0	$0 < 1?$	True → left
3	Leaf	—	—	—	Return 0 (No)

3.7 Two Defects and Their Resolution

The initial implementation contained a single stopping condition — `entropy(y) == 0`. This proved insufficient in two distinct ways. Both are documented in full because both were subtle, both would have survived casual testing, and both are instructive.

3.7.1 Defect 1: Undefined behaviour on an empty subset

Trigger. The split search uses observed feature values as candidate thresholds. If the selected threshold happens to be the *minimum* value present in that column, then no example satisfies `value < threshold` — not even the example the threshold was taken from, since $x < x$ is false. The entire dataset routes to one side and `left_y` is empty.

Failure mechanism. The code then recursed into that empty subset:

1. `entropy(y)` is computed on an empty vector, returning 0.0.
2. The condition `entropy(y) == 0` therefore evaluates to `true`.
3. The code executes `leaf_node.leaf_value = y[0]`.
4. **y is empty. There is no element at index 0.** This is undefined behaviour — in practice either a crash or a silent read of arbitrary memory.

Root cause. The check `entropy(y) == 0` cannot distinguish between two entirely different situations: a genuinely pure node containing many agreeing examples, and an *empty* node containing nothing at all. Both produce zero entropy. The code implicitly assumed the former.

An illustrative framing: “*if a room contains only people wearing red shirts, ring the bell.*” If the bell rings, how many people are in the room? The question is unanswerable — the bell rings both for a room full of people in red, *and* for an empty room, since the condition is vacuously satisfied when there is nobody to contradict it.

Resolution. The empty case is detected in the *parent*, before recursion, where the parent’s labels are still in scope:

```
1 if (left_y.empty()) {
2     Node left_leaf;
3     left_leaf.is_leaf = true;
4     left_leaf.leaf_value = majority_class(y);    // parent's y, not left_y
5     tree.push_back(left_leaf);
6     leaf_node.left_child_index = tree.size() - 1;
7 } else {
8     // normal recursion
9 }
```

The location is deliberate. Guarding inside the child call would prevent the crash but leave nothing meaningful to predict, since that call’s own `y` really is empty. Checking in the parent preserves access to real data.

On the resulting leaf. A leaf created this way predicts a value despite never having seen a single training example of its own. This is worth stating plainly: it is a *fallback*, not a learned rule. Any prediction from such a leaf carries substantially less evidential weight than one from a leaf trained on real examples.

3.7.2 Defect 2: Non-terminating recursion

Trigger. Suppose a feature holds an identical value across every remaining example — common in small subsets deep in a tree. Every candidate threshold equals that same value, so `value < threshold` is false for every example, and *all* examples route to the right.

Failure mechanism. Unlike Defect 1, `right_y` is not empty — it is **identical to the parent’s `y`**. The recursive call therefore receives exactly the problem it was just given.

Table 12: Trace of non-terminating recursion.

Call	Data received	entropy == 0?	Action
1	4 mixed examples	No	Split → all right
2	Same 4 examples	No	Split → all right
3	Same 4 examples	No	Split → all right
⋮	⋮	⋮	Stack overflow

Because the data never shrinks and never becomes pure, no stopping condition is ever satisfied.

Resolution. A degenerate split of this kind always produces exactly zero gain — the single child’s entropy equals the parent’s, and the empty child contributes nothing to the weighted average, so $\text{gain} = H(S) - H(S) = 0$. Testing for this catches the case directly:

```

1 if (best_gain <= 0) {
2     leaf_node.is_leaf = true;
3     leaf_node.leaf_value = majority_class(y);
4     return leaf_node;
5 }
```

Why `<=` rather than `== 0`. The non-strict comparison is correct regardless of floating-point rounding and requires no proof that gain can never be slightly negative. The same reasoning applies to `current_depth >= max_depth`: `>=` remains correct even if the invariant “depth increases by exactly one” is ever broken by a future change, whereas `==` would silently fail.

Table 13: The two defects are independent; neither fix subsumes the other.

	Defect 1	Defect 2
Subset size passed to recursion	0 (empty)	Unchanged (same as parent)
Does <code>entropy == 0</code> fire?	Yes (vacuously)	No (data still mixed)
Failure mode	Immediate UB on <code>y[0]</code>	Unbounded recursion
Fix	<code>left_y.empty()</code> guard	<code>best_gain <= 0</code> guard

These correspond precisely to the second and third ID3 base cases identified in Section 2.6 — a case of theory predicting exactly which defects an incomplete implementation would exhibit.

3.8 Pre-Pruning: Three Stopping Conditions

Three pre-pruning hyperparameters were added, each guarding a different failure mode.

max_depth — structural limit. Caps how many splits may separate the root from any leaf. Requires tracking the current depth, which necessitates a *second* parameter: `max_depth` is a fixed limit supplied by the caller, while `current_depth` changes on every recursive call. The recursive calls pass `current_depth + 1`; the top-level call passes 0 via a default argument.

min_samples_split — statistical reliability. Refuses to split a node holding fewer than this many examples, on the grounds that a pattern observed in two or three examples is as likely coincidence as signal. Note the comparison is `y.size() < min_samples_split`, not `<=`: a node holding exactly the minimum is still permitted to split.

purity_threshold — diminishing returns. Terminates once entropy falls at or below a threshold, without requiring exact purity. The final splits needed to drive entropy from near-zero to exactly zero frequently isolate a single anomalous or mislabelled example — precisely the splits most likely to represent noise.

Table 14: The three conditions measure genuinely different quantities. Each can fire when the others would not.

Scenario	Depth	Samples	Purity	Which fires
Depth 2, 50 examples, $H = 0.9$	no	no	yes	Purity only
Depth 2, 3 examples, $H = 0.0$	no	yes	(pure)	Samples first
Depth 10, 200 examples, $H = 0.7$	yes	no	no	Depth only

Depth is a proxy for structural complexity; sample count for statistical trustworthiness; purity for remaining extractable signal. None subsumes the others, which is why standard libraries expose all three independently.

Placement. All three are checked at the very top of `build_tree`, before the split search. Their order relative to *each other* is immaterial — they are independent guards — but their placement before the expensive search is essential.

3.9 Validation Methodology

Table 15: Verification of core quantities against manual computation.

Quantity	Hand calculation	Implementation	Agreement
$H(S)$, full dataset	0.941	0.940286	✓
Gain(Humidity < 1)	0.151	0.151836	✓

Unit-level verification against hand calculations.

Holdout testing. Rows 3, 7, and 10 were withheld from training:

```
row 3: predicted Yes, actual Yes
row 7: predicted No,  actual No
row 10: predicted No, actual Yes  -- wrong
2/3 correct on holdout
```

Analysis of the failure. Row 10 is {Sunny, Mild, Normal, Strong} with a true label of Yes. Tracing it: Humidity = Normal fails < 1 , routing right; Wind = Strong fails < 1 , routing right; that leaf predicts No.

Inspecting the training data reveals why. The only *other* example with Normal humidity and Strong wind is row 6 ({Rain, Cool, Normal, Strong}), labelled No. With row 10 held out, the tree saw exactly one example of this combination and generalised confidently from it. This is not a defect — it is the expected behaviour of any learner given a single observation, and it illustrates precisely why holdout evaluation is necessary.

Final model selection. Having validated the methodology, the exported tree was retrained on all fourteen rows. This follows standard practice: the holdout exists to validate the *approach*, after which discarding real training data for the deployed model serves no purpose.

3.10 Hyperparameter Sweep

Table 16: Systematic hyperparameter sweep. Each parameter varied while the others were held at permissive values.

Parameter	Values tested	Holdout accuracy
max_depth	1, 2, 3, 4, 100	2/3 for every value
min_samples_split	2, 3, 4, 5, 7	2/3 except 1/3 at 3
purity_threshold	0.0, 0.2, 0.3, 0.5, 0.7, 0.9	2/3 for every value

Interpretation. The result is essentially null, and this is reported honestly rather than presenting a favourable subset.

Three factors explain it. First, the tree naturally terminates around depth 3 regardless, so depth limits above that are inert. Second, no node’s natural entropy exceeded 0.9, so the purity threshold never triggered within the tested range. Third, and most importantly, **a three-example holdout set can only produce four possible accuracy values** (0/3, 1/3, 2/3, 3/3). The measurement resolution is far too coarse to detect anything but the largest effects.

The single deviation — `min_samples_split = 3` degrading accuracy to 1/3 — is instructive: it demonstrates that pre-pruning is a genuine trade-off, capable of removing useful splits as well as noisy ones.

3.11 Which Features the Tree Actually Uses

The final tree tests only **Humidity** and **Outlook**. Temperature and Wind appear nowhere. Because this is initially counter-intuitive, the information gain of every feature was computed at every decision point.

Table 17: Information gain by feature at each decision point. Winners in bold.

	Root	Left child	Right child
Feature	$n = 14$ $H = 0.940$	Humidity=High, $n = 7$ $H = 0.985$	Humidity=Normal, $n = 7$ $H = 0.592$
Outlook	0.102	0.522	0.198
Temperature	0.025	0.020	0.128
Humidity	0.152	0.000 (used)	0.000 (used)
Wind	0.048	0.020	0.198

Findings.

1. **Temperature never competed.** Its gain was lowest of all four at the root, and roughly twenty-six times lower than Outlook’s at the left child. Its exclusion reflects a genuine lack of separating power.
2. **Wind tied exactly with Outlook** at the right child (both 0.198) and lost solely to the tie-breaking rule. The search loop iterates features in index order using a strict `>` comparison, so Outlook (index 0) retains the lead over Wind (index 3). Wind’s exclusion is therefore an artefact of arbitrary tie-breaking, not a quality difference.

3. **Humidity’s gain drops to exactly zero** in both children — the quantitative confirmation that an attribute conveys no further information once split upon.

On whether information is lost. No data is discarded during training: every feature is evaluated against every candidate threshold using all examples available at each node. However, a greedy algorithm evaluates features *individually*, and can in principle miss interactions in which two features are jointly informative but individually weak. Given the magnitude of the gaps observed here, such an interaction is unlikely on this dataset — but the limitation is inherent to greedy tree induction and is noted for completeness.

4 Flattening: Bridging Software and Hardware

4.1 Why Flattening Is Necessary

A C++ tree lives in RAM. It is built at runtime, occupies memory allocated on demand, and its nodes reference one another through indices into a `std::vector` that can grow, shrink, or be reallocated at any moment.

None of this exists in hardware. An FPGA has:

- **No heap** — no dynamic allocation, no `std::vector`
- **No pointers** — no way to follow an arbitrary memory address at runtime
- **No function calls** — and therefore no recursion
- **Fixed physical structure** — every wire and register must be decided before the circuit is built, and cannot change afterwards

The tree must therefore be converted into a form containing only fixed, unchanging data: a table of numbers, of known size, in which the connections between nodes are expressed as **plain integers** rather than references. This conversion is *flattening*.

An analogy: the in-memory tree is like a family tree diagram, with boxes connected by drawn lines. Flattening converts it into a spreadsheet — one row per person, where “who is whose parent” is recorded as a row number in a column, not as a line on a page. The information is identical; the representation is one a machine with no concept of “lines” can consume.

4.2 Designing the Encoding

The encoding was designed on paper before any code was written. Each node becomes one row with six columns.

Table 18: The flattened array encoding. Each node occupies one row of six fields.

Column	Meaning	Meaningful when
<code>feature_idx</code>	Which feature this node tests	Internal nodes only
<code>threshold</code>	The cutoff value compared against	Internal nodes only
<code>left_idx</code>	Row to jump to if the test is true	Internal nodes only
<code>right_idx</code>	Row to jump to if the test is false	Internal nodes only
<code>is_leaf</code>	Flag: 1 = terminal, 0 = internal	Always
<code>leaf_value</code>	The prediction	Leaves only

The don’t-care principle. Every row has all six columns regardless of type, because hardware requires a fixed row width. But an internal node has no prediction, and a leaf has no question — so roughly half of each row is meaningless. These slots are marked **X** (don’t-care) during design, and assigned a concrete placeholder value in the exported file.

node	feature_index	threshold	left_index	right_index	is_leaf	leaf_value
Humidity	2	1	3	4	0	X
Outlook	0	1	5	6	0	X
Wind	3	1	7	8	0	X
Leaf: No	X	X	X	X	1	0
Leaf: Yes	X	X	X	X	1	1
Leaf: Yes	X	X	X	X	1	1
Leaf: No	X	X	X	X	1	0

Figure 2: The flattened array encoding as originally designed by hand in a spreadsheet, before any code was written. Don’t-care fields are marked X. Working the encoding out manually first — rather than deriving it from code — made the distinction between *determined* and *genuinely arbitrary* fields explicit.

Applying the scheme to the final tree yields Table 19.

Table 19: The final tree in flattened form.

Row	Node	feat	thresh	left	right	is_leaf	leaf_val
0	Humidity < 1?	2	1	1	4	0	X
1	Outlook < 1?	0	1	2	3	0	X
2	Leaf	X	X	X	X	1	0
3	Leaf	X	X	X	X	1	1
4	Outlook < 2?	0	2	5	6	0	X
5	Leaf	X	X	X	X	1	1
6	Leaf	X	X	X	X	1	1

Two categories of unknown. During design, a distinction emerged that proved important. Some blanks are *determined* — `left_idx` for row 0 must point at wherever the Outlook node ends up, and there is exactly one correct answer. Others are *genuine don’t-cares* — `leaf_value` for an internal node has no correct answer because nothing will ever read it. Conflating these caused initial confusion; separating them clarified that only the don’t-cares are free choices.

Row numbers are arbitrary; relationships are not. Reordering the table would change every index value, yet the table would remain equally correct provided every `left_idx/right_idx` still pointed at the row its child actually occupies. This realisation is what makes clear that flattening must be done *programmatically* rather than by hand: a program can track “I just placed this node at position 7” reliably, whereas manual bookkeeping is error-prone precisely because the numbers carry no intrinsic meaning.

4.3 `flatten_tree()` — Producing the Table

```

1  std::vector<Node> flatten_tree(const Node& root){
2      std::vector<Node> flattened;
3      if (root.is_leaf) {
4          flattened.push_back(root);
5          return flattened;
6      }
7
8      Node root_copy = root;
```

```

9      flattened.push_back(root_copy);    // this node occupies index 0 of THIS
      list
10
11      std::vector<Node> left_part  = flatten_tree(tree[root.left_child_index]);
12      std::vector<Node> right_part = flatten_tree(tree[root.right_child_index]);
13
14      int left_offset  = flattened.size();           // = 1
15      int right_offset = flattened.size() + left_part.size(); // = 1 + /
      left_part/
16
17      for (Node& n : left_part) {
18          if (!n.is_leaf) {
19              n.left_child_index  += left_offset;
20              n.right_child_index += left_offset;
21          }
22      }
23      for (Node& n : right_part) {
24          if (!n.is_leaf) {
25              n.left_child_index  += right_offset;
26              n.right_child_index += right_offset;
27          }
28      }
29
30      flattened[0].left_child_index  = left_offset;
31      flattened[0].right_child_index = right_offset;
32
33      flattened.insert(flattened.end(), left_part.begin(), left_part.end());
34      flattened.insert(flattened.end(), right_part.begin(), right_part.end());
35
36      return flattened;
37  }

```

Listing 9: Flattening, with index remapping.

Traversal strategy. Unlike `predict()`, which follows a single data-determined path, `flatten_tree` visits **every node unconditionally** — both children, every time. Its structure therefore resembles `print_tree()` rather than `predict()`.

Each recursive call solves a self-contained subproblem: *“return a complete, correctly-indexed flat list of everything in my subtree.”* A leaf’s version is trivial. An internal node’s version is: solve the left subtree’s problem, solve the right subtree’s problem, then combine those two finished lists with itself.

The insert operation. `flattened.insert(flattened.end(), left_part.begin(), left_part.end())` copies every element of `left_part` onto the end of `flattened`. The three arguments specify *where* to insert (at the end) and *what range* to copy. It is equivalent to a loop calling `push_back` for each element.

The offset arithmetic — the crux of the function. Each recursive call returns a list whose indices are correct *relative to itself*, numbered from zero. When two such lists are concatenated behind a parent node, every index inside them becomes wrong by exactly the number of rows now preceding them.

Table 20: Positions shift when subtree lists are merged behind a parent.

	Before merging	After merging
<code>left_part[0]</code> (Outlook)	position 0	position 1
<code>left_part[1]</code> (Leaf)	position 1	position 2
<code>left_part[2]</code> (Leaf)	position 2	position 3

Outlook’s stored `left_child_index` said 1 — correct within `left_part`, where the leaf really was at position 1. After merging, that leaf sits at position 2, so the stored index must be incremented by exactly the amount everything shifted: `left_offset`, which equals 1 (the single row occupied by the parent).

`right_offset` is larger — $1 + |\text{left_part}|$ — because the right subtree is placed after both the parent *and* the entire left subtree.

Finally, the parent’s own indices are set directly: its left child is the first element of `left_part`, which lands precisely at `left_offset`.

Why leaves are skipped in the shifting loops. The `if (!n.is_leaf)` guard exists because a leaf’s child indices are don’t-cares holding no meaningful value. Shifting them would be harmless but pointless.

4.4 The Index-Remapping Defect

An earlier version of `flatten_tree` omitted the offset arithmetic entirely — it simply concatenated the three lists. It compiled, ran without error, and produced a table containing exactly the right seven nodes. It was nevertheless completely broken.

Table 21: Output of the defective flattening. Every internal node’s pointers are wrong, including two self-references.

Row	Node	left	right	Problem
0	Humidity	2	5	Points at a leaf , not the Outlook node
1	Outlook	0	1	Points back at its own parent , and at itself
4	Wind	3	4	Points at itself

Root cause. Each Node was copied verbatim into the new list, including `left_child_index` and `right_child_index` — values that were only ever meaningful as positions within the *original global tree vector*, which was built in a different order by `build_tree`’s recursion. Copying a node to a new location does not update fields describing where it points; those numbers travelled along unchanged, and in the new ordering they addressed entirely unrelated rows.

Consequences had this been deployed. Two distinct failure modes:

- *Row 0’s `left_idx` = 2* pointing at a leaf: the hardware would skip the Outlook test entirely and emit whatever that leaf contained — a silently wrong answer that still *looks* like a valid prediction.
- *Row 1’s `right_idx` = 1* pointing at itself: the hardware would enter an infinite loop, cycling on the same row forever and never reaching a leaf. For certain inputs, no answer would ever be produced.

The second is arguably worse than a crash: on real hardware there is no stack to overflow and no exception to raise. The circuit would simply hang.

Why this defect is instructive. It illustrates a general principle: **an index is only meaningful relative to the container it indexes.** Copying indexed data between containers silently invalidates every stored index unless they are explicitly recomputed. The defect was invisible to a compiler, produced no warning, and yielded output that looked structurally plausible — it was caught only by manually cross-referencing each pointer against the row it actually addressed.

4.5 export_vh() — Emitting the Hardware Header

```

1 void export_vh(const std::vector<Node>& flat, int num_features,
2               const std::string& filepath){
3     int n = (int)flat.size();
4
5     int feature_bits = std::max(1,
6                               (int)std::ceil(std::log2(std::max(2, num_features))));
7     int row_bits = std::max(1,
8                             (int)std::ceil(std::log2(std::max(2, n))));
9
10    int max_threshold = 0;
11    for (auto& nd : flat)
12        if (!nd.is_leaf)
13            max_threshold = std::max(max_threshold, (int)nd.threshold);
14    int threshold_bits = std::max(1,
15                                  (int)std::ceil(std::log2(std::max(2, max_threshold + 1))));
16
17    std::ofstream file(filepath);
18    file << "'ifndef DECISION_TREE_VH\n#define DECISION_TREE_VH\n\n";
19    file << "localparam int NUM_NODES = " << n << ";\n";
20    file << "localparam int FEATURE_BITS = " << feature_bits << ";\n";
21    file << "localparam int THRESHOLD_BITS = " << threshold_bits << ";\n";
22    file << "localparam int ROW_BITS = " << row_bits << ";\n\n";
23
24    auto pack_field = [&](const std::string& name, int bits, auto getter){
25        file << "localparam [" << (bits * n - 1) << ":0] " << name << " = {";
26        for (int i = n - 1; i >= 0; i--) { // row n-1 first, row 0 last
27            file << bits << "'d" << getter(flat[i]);
28            if (i != 0) file << ", ";
29        }
30        file << "};\n";
31    };
32
33    pack_field("feature_idx_packed", feature_bits,
34              [](const Node& nd){ return nd.is_leaf ? 0 : nd.feature_index; });
35    pack_field("threshold_packed", threshold_bits,
36              [](const Node& nd){ return nd.is_leaf ? 0 : (int)nd.threshold; });
37    pack_field("left_idx_packed", row_bits,
38              [](const Node& nd){ return nd.is_leaf ? 0 : nd.left_child_index; });
39    pack_field("right_idx_packed", row_bits,
40              [](const Node& nd){ return nd.is_leaf ? 0 : nd.right_child_index; });
41    pack_field("is_leaf_packed", 1,
42              [](const Node& nd){ return nd.is_leaf ? 1 : 0; });
43    pack_field("leaf_value_packed", 1,
44              [](const Node& nd){ return nd.is_leaf ? nd.leaf_value : 0; });
45
46    file << "\n`endif\n";
47    file.close();
48 }

```


Listing 10: Header export with automatic bit-width computation.

The `pack_field` lambda. Rather than repeating near-identical output code six times, a lambda accepts a field name, its bit-width, and an accessor describing how to extract that field from a `Node`. The `[&]` capture allows it to reference `file` and `n` from the enclosing scope.

Note the loop runs **backwards**, from row $n - 1$ down to row 0. Verilog concatenation places the first-listed item at the most significant bit position, so emitting rows in reverse places **row 0 at the least significant end** — which is what makes the slicing arithmetic in the hardware module (`row * WIDTH +: WIDTH`) correct.

Column-major layout. Rather than one array per node, the export produces **one array per field**, each holding that field’s value for all nodes. Reconstructing node k requires reading index k from all six arrays. This organisation maps naturally onto hardware, where a lookup table indexed by the current row is a primitive operation.

Automatic bit-width computation. The three `ceil(log2(...))` expressions compute the minimum bits each field requires, derived from the actual exported tree. The nested `std::max(2, ...)` guards against $\log_2(1) = 0$ and $\log_2(0) = -\infty$ for degenerate inputs. Because these are computed rather than hardcoded, the exporter adapts automatically to any dataset.

Choice of don’t-care value. The accessors emit 0 for don’t-care slots rather than -1 . This was deliberate: -1 requires a signed representation, and a sign bit costs an extra bit in *every* entry of the field. Because `is_leaf` already indicates when the other fields should be ignored, the placeholder need not be distinguishable from a real value — so an in-range non-negative value is strictly cheaper. This single decision reduced `threshold` from a 3-bit signed field to a 2-bit unsigned one.

```
1  'ifndef DECISION_TREE_VH
2  'define DECISION_TREE_VH
3
4  localparam int NUM_NODES = 7;
5  localparam int FEATURE_BITS = 2;
6  localparam int THRESHOLD_BITS = 2;
7  localparam int ROW_BITS = 3;
8
9  localparam [13:0] feature_idx_packed = {2'd0,2'd0,2'd0,2'd0,2'd0,2'd0,2'd2};
10 localparam [13:0] threshold_packed = {2'd0,2'd0,2'd2,2'd0,2'd0,2'd1,2'd1};
11 localparam [20:0] left_idx_packed = {3'd0,3'd0,3'd5,3'd0,3'd0,3'd2,3'd1};
12 localparam [20:0] right_idx_packed = {3'd0,3'd0,3'd6,3'd0,3'd0,3'd3,3'd4};
13 localparam [6:0] is_leaf_packed = {1'd1,1'd1,1'd0,1'd1,1'd1,1'd0,1'd0};
14 localparam [6:0] leaf_value_packed = {1'd1,1'd1,1'd0,1'd1,1'd0,1'd0,1'd0};
15
16 'endif
```

Listing 11: The generated header file.

Reading this file. The notation `2'd0` denotes “the decimal value 0, represented in 2 bits”. The braces perform concatenation. Thus `feature_idx_packed` is a single 14-bit constant (7 nodes $\times$ 2 bits) whose least significant two bits hold `2'd2` — row 0’s feature index, corresponding to Humidity. The `'ifndef/'define/'endif` wrapper is an **include guard**, preventing duplicate declarations.

4.6 Bit-Width Analysis

Table 22: Minimum bit-width per field, derived from first principles.

Field	Range	Bits	Justification
<code>feature_idx</code>	0–3	2	4 features; $2^2 = 4$
<code>threshold</code>	0–2	2	Largest encoded value is 2; $2^2 = 4 \geq 3$
<code>left_idx</code>	0–6	3	7 nodes; $2^3 = 8 \geq 7$
<code>right_idx</code>	0–6	3	As <code>left_idx</code>
<code>is_leaf</code>	0–1	1	Boolean flag
<code>leaf_value</code>	0–1	1	Binary classification
Total per node		12 bits	

An important consequence of the don’t-care decision emerges here. Had -1 been retained as the placeholder, `threshold` would have needed to represent $\{-1, 0, 1, 2\}$, requiring a **3-bit signed** field rather than a 2-bit unsigned one. The apparently trivial choice of placeholder value therefore directly determines hardware cost.

On headroom. These widths are exact for this specific tree. Supporting up to 15 nodes, for instance, would require 4 index bits ($2^4 = 16 \geq 15$). The exporter computes these automatically, but the *hardware module* currently hardcodes them — a limitation discussed in Section 11.

5 The Toolchain

This section documents every tool used in the hardware phase, why it was chosen, how it was installed, and what each command actually does. It is included in full because toolchain friction consumed a substantial proportion of the project’s time, and because reproducibility requires it.

5.1 Why Ubuntu, and What WSL Is

The problem. Hardware design tools are overwhelmingly developed for Linux. Windows builds exist but frequently lag behind, and this project encountered exactly that: the most readily available prebuilt Windows package of Icarus Verilog was **version 10.1.1**, whereas the SystemVerilog constructs used here — notably `always_comb` — require **version 11 or later** for reliable support. Building from source on Windows was possible in principle but required assembling a full MinGW toolchain plus `bison`, `flex`, `gperf`, `autoconf`, and `make`.

The solution: WSL. The Windows Subsystem for Linux runs a genuine Linux environment *inside* Windows, without dual-booting. Windows remains fully usable; a Linux terminal simply becomes available alongside it. Critically, WSL can access the Windows filesystem (mounted under `/mnt/c/...`), so files move freely between environments.

Inside WSL, installing a current Icarus Verilog reduces to a single command. This is the entire reason WSL was adopted: it converts a multi-step source build into a one-line package installation.

```
wsl --install
```

Listing 12: WSL installation, from an Administrator PowerShell.

On the machine used here, the first attempt failed.

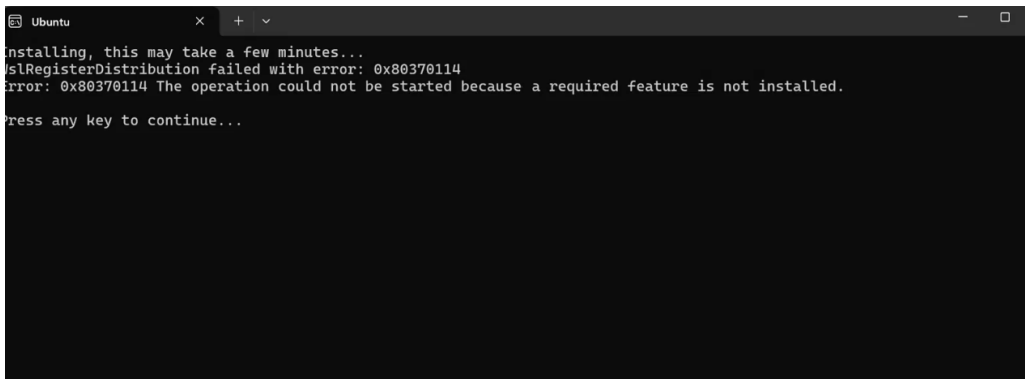


Figure 3: The initial WSL registration failure. Error 0x80370114 indicates that required Windows optional features were not enabled — a common obstacle when installing WSL on a machine where virtualisation support has not previously been activated.

Resolution. Two Windows optional features had to be enabled manually via *Turn Windows features on or off*:

- **Windows Subsystem for Linux**
- **Virtual Machine Platform**

followed by a reboot. Had this not resolved it, the next candidate would have been hardware virtualisation disabled in the system firmware — a BIOS/UEFI setting outside Windows’ control.

A diagnostic note worth recording. Before this was resolved, `sudo apt install yosys` was typed into a Windows Command Prompt rather than the Ubuntu terminal, producing “*Sudo is disabled on this machine*”. This is Windows 11’s own, unrelated `sudo` feature responding — not Linux’s. The distinction matters: `C:\Users\Name>` indicates Windows; `user@MACHINE:~$` indicates Linux.

5.2 Icarus Verilog — Simulation

What it does. Icarus Verilog compiles Verilog/SystemVerilog and *simulates* it — executing the described logic in software to check that it behaves correctly. It does **not** convert designs into hardware; that is synthesis.

```
sudo apt update
sudo apt install iverilog gtkwave
```

Listing 13: Installing the simulation tools.

Known limitations encountered. Two genuine parser limitations shaped the design:

1. **Array-literal assignment (`'{...}`) to `localparam` arrays is unsupported.** The natural encoding — one array per field — fails to parse. This was confirmed by reducing it to a minimal three-line test case, ruling out any fault in the surrounding code. The workaround was to pack each field into a single wide vector and slice it.
2. **Unpacked array ports are parsed by Icarus but not by Yosys.**

5.3 GTKWave — Waveform Visualisation

What it does. A simulation produces a *value change dump* (`.vcd`) file recording every signal’s value at every point in simulated time. GTKWave renders this as a visual timeline — conceptually an oscilloscope for digital signals.

Why it matters. Text output from `$display` reports only final results: “row 7 got 0, expected 1”. It cannot show **why**. A waveform shows every internal signal evolving step by step, making it possible to see precisely where a computation diverged from expectation.

```
1 $dumpfile("waves.vcd");
2 $dumpvars(0, testbench);
```

Listing 14: Enabling waveform capture.

`$dumpfile` names the output file. `$dumpvars(0, testbench)` records every signal within the `testbench` module and, because the depth argument is 0, everything nested inside it — including the instantiated design under test.

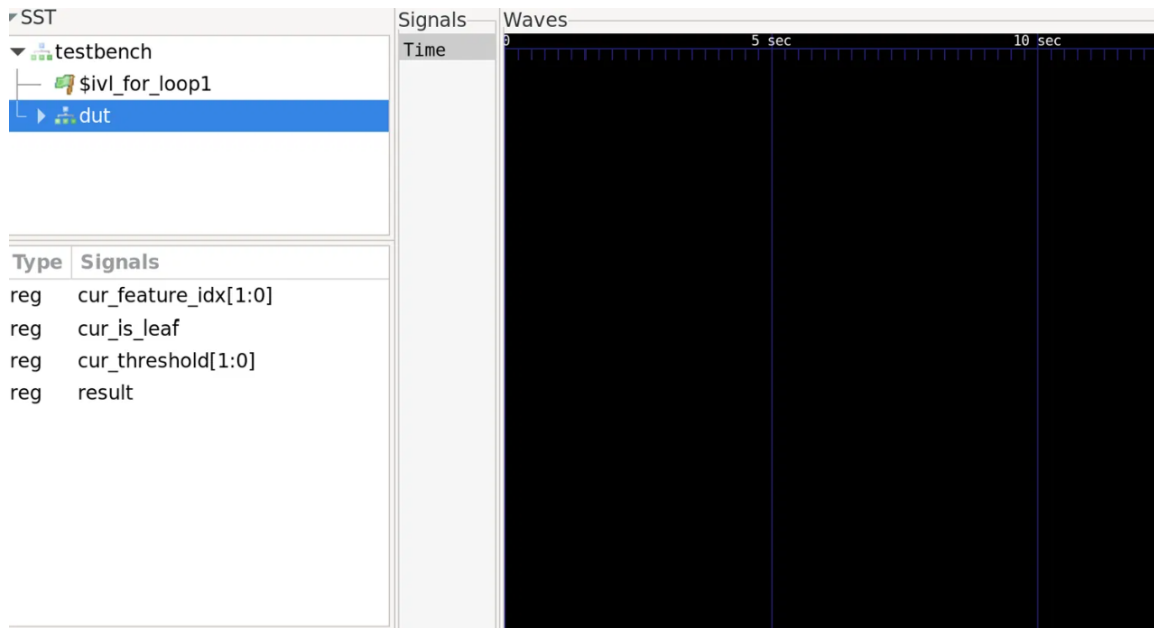


Figure 4: The GTKWave signal browser on first opening. The upper-left pane shows the design hierarchy (`testbench` containing the instantiated `dut`); the lower-left pane lists the signals available within the selected scope. The waveform pane is initially empty — signals must be explicitly added before they are displayed.



Figure 5: Simulation waveforms for the fourteen-vector testbench. The **result** signal (top trace) transitions between logic levels as successive test vectors are applied, each occupying one time unit. Because the design under test is purely combinational, these transitions occur immediately upon each input change with no clock edge required — the visual confirmation that the tree resolves “in one pass”.

Observations from this project. Signals were successfully displayed and **result** was observed transitioning between test vectors, confirming the simulation was genuinely exercising the design. However, the internal **row** and **done** **unpacked arrays did not appear** in the signal browser — a known characteristic of how Icarus and GTKWave handle array-typed signals, which are treated differently from scalar signals in the VCD format. Since all fourteen test vectors passed, no waveform debugging was actually required; the exercise served to establish familiarity with the tool for future use.

5.4 Yosys — Synthesis

What it does, and how it differs from simulation. This distinction is central. Icarus *simulates*: it pretends to be the circuit and checks the behaviour is right. Yosys *synthesises*: it converts the design description into an actual netlist of logic gates — the specification of a physical circuit.

The software analogy is a compiler. Writing `if (x < y)` in C++ produces machine instructions the programmer never sees. Yosys performs the equivalent transformation for hardware.

Table 23: Yosys versus Vivado for this project’s requirements.

	Vivado	Yosys
Install size	Multiple gigabytes	~30 MB via <code>apt</code>
Licensing	Registration required	Open source, none
Install time	Hours (download-dominated)	Seconds
Xilinx primitive mapping	Exact, vendor-authoritative	Approximate but genuine
Timing analysis	Full, accurate	Not available in basic flow

Since no physical board was available, the only capability lost is precise timing analysis. The substitution is documented as a deliberate, reasoned trade-off rather than an omission.

Limitation encountered. Yosys’s Verilog frontend cannot parse **unpacked array ports**:

```
1 input logic [1:0] features [0:3] // parses in Icarus, fails in Yosys
```

The module was therefore restructured to accept a single packed vector, with individual features extracted internally by slicing. The logic is unchanged; only the interface packaging differs. This reflects a general principle: **simulation tools and synthesis tools accept overlapping but non-identical language subsets**, and a design intended for both must target their intersection.

5.5 Command Reference

Table 24: Every distinct command used during the hardware phase.

Command	Purpose
<i>Filesystem navigation</i>	
<code>mkdir ~/decision_tree</code>	Create a working directory. <code>~</code> denotes the home directory
<code>cd ~/decision_tree</code>	Change into that directory; relative paths resolve from here
<code>ls</code>	List directory contents
<code>pwd</code>	Print the current directory — diagnosed a case where <code>nano</code> created a new file because the working directory was wrong
<code>cat file</code>	Print a file’s contents
<code>tail -5 file</code>	Print the last five lines — the diagnostic that located a truncated file
<code>cat -A file</code>	Print contents including invisible characters — used to rule out Windows line endings

Command	Purpose
<code>head -c 50 file xxd</code>	Display the first fifty bytes as hexadecimal — used to rule out a byte-order mark
<i>File creation</i>	
<code>nano file</code>	Terminal text editor. Ctrl+O saves; Ctrl+X exits
<code>cat > file « 'EOF' ... EOF</code>	Write a file directly from the shell, no editor. Adopted after nano produced a truncated file; the quoted 'EOF' prevents shell interpretation of the content
<i>Search</i>	
<code>find / -iname "cells_sim.v"</code>	Search the entire filesystem by name. Impractically slow; interrupted with Ctrl+C
<code>dpkg -L yosys grep sim</code>	Ask the package manager which files yosys installed. Located the target in under a second
<i>Simulation</i>	
<code>iverilog -g2012 -o sim m.sv tb.sv</code>	Compile sources into an executable simulation. -g2012 selects the SystemVerilog-2012 standard, required for always_comb/always_ff
<code>vvp sim</code>	Execute the compiled simulation
<code>gtkwave waves.vcd</code>	Open the waveform viewer
<code>pkill gtwave / killall -9 gtwave</code>	Terminate an unresponsive process; -9 forces immediate termination
<i>Synthesis (shell)</i>	
<code>yosys -s script.js</code>	Run Yosys with a script rather than interactively
<i>Synthesis (script directives)</i>	
<code>read_verilog -sv file.sv</code>	Parse the design as SystemVerilog
<code>hierarchy -top module</code>	Identify the top-level module — analogous to nominating main()
<code>proc</code>	Convert procedural blocks into an internal netlist
<code>opt</code>	Apply optimisation passes, removing redundant logic
<code>synth -top module</code>	Generic technology-independent synthesis
<code>synth_xilinx -top module</code>	Synthesis targeting Xilinx 7-series primitives
<code>stat</code>	Print resource statistics — the source of every cell count in this paper
<code>write_verilog -noattr out.v</code>	Write the gate-level netlist to a file

6 Hardware Implementation in SystemVerilog

6.1 The Combinational Module

```

1  'include "decision_tree.vh"
2
3  module decision_tree (
4      input  logic [7:0] features_packed,
5      output logic      result
6  );
7
8      logic [2:0] row    [0:NUM_NODES];
9      logic      done    [0:NUM_NODES];

```

```

10     logic [1:0] cur_feature_idx;
11     logic [1:0] cur_threshold;
12     logic      cur_is_leaf;
13     logic [1:0] this_feature_val;
14
15     always_comb begin
16         row[0] = 3'd0;
17         done[0] = 1'b0;
18
19         for (int i = 0; i < NUM_NODES; i = i + 1) begin
20             cur_feature_idx = feature_idx_packed[row[i]*2 +: 2];
21             cur_threshold   = threshold_packed[row[i]*2 +: 2];
22             cur_is_leaf     = is_leaf_packed[row[i]];
23             this_feature_val = features_packed[cur_feature_idx*2 +: 2];
24
25             if (done[i] || cur_is_leaf) begin
26                 row[i+1] = row[i];
27                 done[i+1] = 1'b1;
28             end else begin
29                 if (this_feature_val < cur_threshold)
30                     row[i+1] = left_idx_packed[row[i]*3 +: 3];
31                 else
32                     row[i+1] = right_idx_packed[row[i]*3 +: 3];
33                 done[i+1] = 1'b0;
34             end
35         end
36
37         result = leaf_value_packed[row[NUM_NODES]];
38     end
39
40 endmodule

```

Listing 15: The combinational tree-walk module.

6.1.1 Module Declaration and Ports

A **module** is the hardware equivalent of a function definition, and its **ports** are its interface. Unlike a function, however, a module is not “called” — it is permanently existing circuitry whose outputs continuously reflect its inputs.

`input logic [7:0] features_packed` declares an 8-bit input: four features at two bits each, packed into one vector with feature 0 occupying the least significant bits.

Why packed rather than four separate wires. This was originally written as `logic [1:0] features [0:3]` — an unpacked array, closely mirroring the C++ `row`. Two considerations forced the change. First, Yosys cannot parse unpacked array ports. Second, and more fundamentally, the array form obscures a physical truth: hardware wires are **fixed and physical**. A `std::vector` can grow, shrink, and reallocate; a wire cannot.

6.1.2 `always_comb` — Combinational Behaviour

`always_comb` declares that the enclosed logic is purely combinational: its outputs are a direct function of its inputs, recomputed **immediately and continuously** whenever any input changes.

This differs fundamentally from a C++ function. `predict(root, row)` executes only when explicitly called. An `always_comb` block is not called — it is a standing rule, permanently active. Change `features_packed`, and `result` updates automatically with no invocation, no clock, and no waiting. This is precisely what the specification’s phrase “*the whole tree resolves in one pass*” means.

6.1.3 The Loop as a Recursion Substitute

`predict()` recurses. Hardware cannot: a circuit is a fixed physical structure and cannot instantiate copies of itself at runtime.

The `for` loop is the substitute. Critically, **it is not a runtime loop** — during synthesis it is *unrolled*, producing `NUM_NODES` physically distinct copies of the comparison logic, chained together. Iteration i computes the state after i hops; iteration $i + 1$ consumes that result.

Because the loop count is fixed at `NUM_NODES = 7`, it always executes seven times regardless of how deep the actual answer lies. This is safe because **a tree of n nodes can never require more than n hops** to reach a leaf.

6.1.4 The done Flag

Since the loop cannot terminate early, a mechanism is needed to “freeze” once a leaf is reached. Two conditions are combined:

- `cur_is_leaf` — a leaf has been reached *on this iteration*
- `done[i]` — a leaf was reached on some *earlier* iteration

The second is essential. Without it, once a leaf were reached, the following iteration would treat that leaf’s row as an internal node, reading its don’t-care placeholders and computing a nonsensical next row. With it, `row` simply copies forward unchanged, leaving `row[NUM_NODES]` correctly holding the leaf’s index.

6.1.5 Bit-Slicing

The `+:` operator performs an **indexed part-select**: `[start +: width]` extracts `width` bits beginning at bit position `start`. Since each node’s `feature_idx` occupies exactly two bits, node k ’s field begins at bit $2k$.

Table 25: Bit-slice addressing for the packed `feature_idx` field.

<code>row[i]</code>	Start bit	Bits extracted	Meaning
0	0	0–1	Node 0’s feature index
1	2	2–3	Node 1’s feature index
4	8	8–9	Node 4’s feature index

This is the hardware equivalent of `flat[row].feature_index` — but where `std::vector` performs the address arithmetic invisibly, hardware requires it written explicitly.

6.1.6 Correspondence to predict()

Table 26: Line-by-line correspondence between the C++ reference and the hardware implementation.

predict() (C++)	decision_tree (SystemVerilog)
if (node.is_leaf) return node.leaf_value; row[node.feature_index]	if (done[i] cur_is_leaf) → freeze; result = leaf_value_packed[...] features_packed[cur_feature_idx*2 +: 2]
< node.threshold	< cur_threshold
predict(tree[node.left_child_in- dex], row)	row[i+1] = left_idx_packed[row[i]*3 +: 3]
predict(tree[node.right_child_- index], row)	row[i+1] = right_idx_packed[row[i]*3 +: 3]
Recursion	Unrolled fixed-count loop plus done flag

Every line of decision logic maps directly. The only structural difference is the recursion-to-unrolled-loop transformation, forced by hardware’s inability to self-invoke.

6.2 The Self-Checking Testbench

```

1  module testbench;
2
3      logic [7:0] features_packed;
4      logic      result;
5
6      decision_tree dut (
7          .features_packed(features_packed),
8          .result(result)
9      );
10
11     logic [1:0] test_features    [0:13][0:3];
12     logic      expected_results [0:13];
13
14     initial begin
15         $dumpfile("waves.vcd");
16         $dumpvars(0, testbench);
17
18         test_features[0][0]=2'd0; test_features[0][1]=2'd0;
19         test_features[0][2]=2'd0; test_features[0][3]=2'd0;
20         expected_results[0]=1'd0;
21         // ... rows 1 through 13 ...
22
23         for (int row = 0; row < 14; row = row + 1) begin
24             features_packed = {test_features[row][3], test_features[row][2],
25                               test_features[row][1], test_features[row][0]};
26             #1;
27             if (result === expected_results[row])
28                 $display("PASS row %0d -> %0d", row, result);
29             else
30                 $display("FAIL row %0d -> got %0d, expected %0d",
31                           row, result, expected_results[row]);
32         end
33
34         $finish;
35     end

```

```

36
37 endmodule

```

Listing 16: The fourteen-vector self-checking testbench (data assignments abridged).

A testbench has no ports. Unlike the design under test, it does not connect to anything external — it *is* the external world, standing in for whatever would eventually drive the circuit. It exists purely within simulation and is never synthesised.

Instantiation. `decision_tree dut (...)` creates an instance of the design, conventionally named `dut` (“device under test”). The `.port(signal)` syntax connects the testbench’s signals to the module’s ports.

Data–logic separation. An initial version tested two vectors by writing out the assign–wait–check sequence twice. Extending that to fourteen would have required fourteen near-identical blocks. The final version stores test data in arrays and applies a single loop — the same principle that motivates extracting a shared helper from duplicated code. Adding a fifteenth vector now requires one data line and one bound change.

Why #1. Although `always_comb` responds “immediately”, a simulator still requires a nonzero time step for new values to propagate before they are read.

Why === rather than ==. The four-state equality operator `===` compares exactly, including the `x` (unknown) and `z` (high-impedance) states hardware signals can hold. The ordinary `==` returns `x` — neither true nor false — when either operand is unknown, which in a testbench would silently mask genuine failures.

Ground truth. The `expected_results` array holds the outputs of the C++ `predict()` function on the *same* tree. The testbench therefore verifies **equivalence between hardware and software**, not accuracy against real labels. Accuracy was established separately during the holdout evaluation; this stage answers a narrower question: *does the hardware reimplement faithfully reproduce the reference behaviour?*

PASS row 0 -> 0	PASS row 7 -> 0
PASS row 1 -> 0	PASS row 8 -> 1
PASS row 2 -> 1	PASS row 9 -> 1
PASS row 3 -> 1	PASS row 10 -> 1
PASS row 4 -> 1	PASS row 11 -> 1
PASS row 5 -> 1	PASS row 12 -> 1
PASS row 6 -> 1	PASS row 13 -> 1

Listing 17: Simulation output — all fourteen vectors pass.

7 Optimisation Studies

7.1 Overview of Strategies

Two independent optimisation strategies were pursued, targeting different aspects of hardware cost. Table 27 summarises them before each is examined in detail.

Table 27: Optimisation strategies applied, with mechanism and measured outcome.

Strategy	Mechanism	Cost incurred	Measured saving
Bit-width minimisation	Compute each field’s width from the tree’s actual value ranges rather than defaulting to a generic width	None — functionally identical	43% (174 → 99)
Pipelining	Replace the unrolled parallel tree walk with one level per clock cycle, reusing a single stage	3 cycles latency; 6 flip-flops	53% (99 → 46)
Technology mapping	Target Xilinx primitives, allowing constant-folding of the fixed tree structure into lookup tables	Device-specific; not portable	1 LUT4 total

7.2 Study 1 — Bit-Width Minimisation

Hypothesis. Using the minimum bit-width each field genuinely requires, rather than a generic default, meaningfully reduces synthesised hardware.

Method. Two functionally identical designs were synthesised. The only difference was field width; the tree data, the algorithm, and every line of decision logic were identical. This isolates bit-width as the sole independent variable.

Table 28: Experimental conditions for the bit-width study.

Field	Minimal version	Oversized version
<code>feature_idx</code>	2 bits	8 bits
<code>threshold</code>	2 bits	8 bits
<code>left_idx / right_idx</code>	3 bits	8 bits
Input port	8 bits total	32 bits total
row register	3 bits	8 bits

Table 29: Synthesis results for the bit-width study.

Version	Cells	Wires	Wire bits
Oversized (8-bit fields)	174	187	283
Minimal (calculated widths)	99	104	115
Reduction	43%	44%	59%

$$\text{Reduction} = \frac{174 - 99}{174} \times 100\% \approx \mathbf{43\%} \quad (12)$$

Interpretation. Wider fields require wider comparators, wider multiplexers, and wider slicing logic throughout. The saving is not incidental — it is the direct consequence of the $\lceil \log_2(\cdot) \rceil$ calculations performed in `export_vh`, propagated through every downstream operation.

7.3 Pipelining — Theory

The combinational design resolves the entire tree walk in one continuous propagation. For a small tree this is ideal. For a large one it becomes problematic: because the loop unrolls into physically chained logic, a 1,000-node tree would produce a chain through which signals must propagate *entirely* before any output is valid. Propagation delay accumulates along that chain, and the maximum achievable clock rate of any surrounding system falls accordingly.

Clocks. A clock is a signal alternating between 0 and 1 at a fixed, regular rate. It carries no data; its sole function is to mark discrete instants in time.

Registers. A register stores a value and updates it **only** at a clock edge — conventionally the rising edge, when the clock transitions $0 \rightarrow 1$. Between edges it holds its value regardless of input activity. This is the element entirely absent from the combinational design: a means of remembering something from one instant to the next.

```
1 always_ff @(posedge clk) begin
2     row <= next_row;
3 end
```

Listing 18: The registered assignment idiom.

- `always_ff` — sequential (clocked) logic, as opposed to `always_comb`
- `@(posedge clk)` — active only on the rising clock edge
- `<=` — nonblocking assignment, the convention within clocked blocks

The trade-off. For a single inference, pipelining is *slower* — several cycles rather than one instantaneous resolution. Two compensating benefits arise:

1. **Each cycle performs less work.** The critical path shrinks to one level’s worth of comparison rather than the entire tree, permitting a higher clock frequency.
2. **Throughput can exceed latency.** With appropriate extension, a second input can enter the pipeline while the first is still progressing — multiple inferences in flight simultaneously, like an assembly line.

7.4 Pipelined Implementation

```
1 'include "decision_tree.vh"
2
3 module decision_tree_pipelined (
4     input  logic      clk,
5     input  logic      rst,
6     input  logic [7:0] features_packed,
7     output logic      result,
8     output logic      valid
9 );
10
11     logic [2:0] row;
12     logic      done;
13
14     always_ff @(posedge clk) begin
15         if (rst) begin
16             row <= 3'd0;
17             done <= 1'b0;
18             valid <= 1'b0;
19         end else if (!done) begin
20             if (is_leaf_packed[row]) begin
21                 result <= leaf_value_packed[row];
22                 done <= 1'b1;
23             end
24         end
25     end
```

```

23         valid  <= 1'b1;
24     end else begin
25         if (features_packed[feature_idx_packed[row*2 +: 2]*2 +: 2]
26             < threshold_packed[row*2 +: 2])
27             row <= left_idx_packed[row*3 +: 3];
28         else
29             row <= right_idx_packed[row*3 +: 3];
30     end
31 end
32 end
33
34 endmodule

```

Listing 19: The pipelined tree-walk module.

Table 30: Structural differences between the two architectures.

Aspect	Combinational	Pipelined
row	Array [0:NUM_NODES] — every step’s state simultaneously	Single register — only the <i>current</i> position
Timing	<code>always_comb</code> , no clock	<code>always_ff @(posedge clk)</code>
Loop	Unrolled seven times in hardware	None — one step per clock cycle
New ports	—	<code>clk</code> , <code>rst</code> , <code>valid</code>

The `row` array collapsing to a single register is the essential change. The combinational version required simultaneous hardware for all seven possible steps; the pipelined version reuses one step’s worth of logic across successive cycles, and needs to remember only where it currently is.

New signals. `rst` is an external reset returning the module to the root. `valid` is an output flag indicating the result is meaningful — necessary because the answer is no longer instantaneous, so an external consumer must know when to trust `result`. `done` is an internal flag preventing further movement once a leaf is reached.

`done` and `valid` assert simultaneously in this design and are therefore momentarily redundant. They are kept separate because they answer conceptually distinct questions — “*have I internally finished?*” versus “*is my output trustworthy?*”

7.5 The Clocked Testbench

```

1  always #5 clk = ~clk;
2
3  task automatic run_case(input logic [1:0] o, t, h, w,
4                          input logic expected, input string label);
5      int cycles;
6      rst = 1;
7      features_packed = {w, h, t, o};
8      @(posedge clk);
9      #1;
10     rst = 0;
11     cycles = 0;
12     while (!valid && cycles < 10) begin
13         @(posedge clk);

```

```

14     #1;
15     cycles = cycles + 1;
16 end
17 if (result == expected)
18     $display("PASS %s -> %0d (took %0d cycles)", label, result, cycles);
19 else
20     $display("FAIL %s -> got %0d, expected %0d", label, result, expected)
21     ;
22 endtask

```

Listing 20: Clock generation and the test task.

`always #5 clk = ~clk;` generates the clock: every five time units, invert it, producing a square wave of period ten.

A significant timing subtlety. The `#1` following each `@(posedge clk)` is essential, and its absence caused a genuine failure during development. Reading a register **immediately** after a clock edge returns the *old* value, because nonblocking assignments (`<=`) are scheduled to update in a distinct simulation phase occurring after the edge itself.

The first attempt omitted these delays and produced:

```

FAIL Rain/Mild/High/Weak -> got x, expected 1
FAIL Sunny/Mild/High/Weak -> got x, expected 0

```

`x` denotes an unknown value — the registers appeared never to have been assigned. Isolating the behaviour in a minimal four-line register test confirmed the cause was the read timing, not the design. Adding `#1` after each edge resolved it entirely.

This is a well-documented characteristic of Verilog’s event scheduling and a classic source of confusion when moving from combinational to sequential design. Understanding *why* it occurs — the distinction between blocking and nonblocking assignment semantics — is genuinely load-bearing knowledge.

```

PASS Rain/Mild/High/Weak -> 1 (took 3 cycles)
PASS Sunny/Mild/High/Weak -> 0 (took 3 cycles)

```

Listing 21: Pipelined simulation output.

The “3 cycles” figure is the empirical confirmation that pipelining is genuinely occurring: the same answer the combinational design produced instantaneously now requires three clock cycles, one per tree level traversed.

7.6 Architectural Comparison

Table 31: Combinational versus pipelined, measured.

Metric	Combinational	Pipelined
Logic cells	99	46
Flip-flops	0	6
Latency (single inference)	1 propagation delay	3 clock cycles
Critical path	Full tree depth	One level
Throughput potential	1 per propagation	1 per cycle (with extension)
Cell reduction		53%

Interpretation. The pipelined design requires fewer than half the logic cells. This follows directly from the architectural difference: the combinational version instantiates hardware for

all seven potential steps in parallel, whereas the pipelined version instantiates one step’s worth and reuses it across cycles.

The flip-flop count rising from zero to six is the visible cost of that reuse — physical storage is now required to remember state between cycles. In the synthesis output these appear as `$_SDFFE_PPOP_`, `$_SDFFF_PP0_`, and `$_DFFE_PP_` primitives, entirely absent from the combinational netlist.

On maximum frequency. The specification also requested a maximum frequency comparison. Yosys’s `stat` pass reports resource counts but not timing estimates, which require technology-specific delay models unavailable in the basic open-source flow. This is recorded as a genuine gap rather than estimated speculatively.

8 Synthesis and Post-Synthesis Verification

8.1 Xilinx-Targeted Synthesis

Generic synthesis maps a design onto abstract logic gates. **Technology-targeted** synthesis maps it onto the specific primitives a particular manufacturer’s chips actually contain.

On Xilinx. Xilinx (now part of AMD) is a major FPGA manufacturer. Their devices are built from specific components with specific names — most importantly the **LUT** (Look-Up Table), a small memory implementing *any* boolean function of its inputs by storing that function’s complete truth table. A LUT4 has four inputs and stores 16 bits: one output value per possible input combination.

Xilinx was targeted because it is a widely deployed family, because the Digilent Basys 3 (a common educational board) uses a Xilinx Artix-7, and because Vivado — the toolchain originally specified — targets Xilinx devices.

```
=== decision_tree ===

Number of wires:           10
Number of wire bits:       32
Number of public wires:    7
Number of public wire bits: 19
Number of cells:           10
  IBUF                     8
  LUT4                     1
  OBUF                     1

Estimated number of LCs:    1
```

Listing 22: Xilinx synthesis result.

Table 32: Resource utilisation across all synthesis targets.

Design	Target	Cells	Breakdown
Oversized combinational	Generic	174	46 ANDNOT, 43 MUX, 42 OR, ...
Minimal combinational	Generic	99	25 MUX, 20 ANDNOT, 18 OR, ...
Pipelined	Generic	46	12 ANDNOT, 7 MUX, 6 flip-flops, ...
Minimal combinational	Xilinx	10	1 LUT4 , 8 IBUF, 1 OBUF

Why the design collapses so dramatically. This result initially appeared implausible for a seven-node tree traversal, and was therefore investigated rather than accepted.

The explanation is that all six packed arrays are `localparam` — **compile-time constants**. The tree structure is entirely fixed before synthesis begins. Consequently, nothing in the traversal logic actually varies at runtime: the only variable input is `features_packed`.

Synthesis is therefore free to evaluate the entire traversal symbolically and reduce it to its minimal boolean form — a function of eight input bits producing one output bit. For this tree, that function proved simple enough to fit within a single 4-input LUT, having determined that only four of the eight input bits (those corresponding to the Humidity and Outlook features actually tested) influence the result at all.

This is not an artefact or an error. It is constant folding operating at full strength, and it reflects a genuine property of the design: **because the tree is fixed at synthesis time, its traversal costs almost nothing in hardware.**

8.2 Post-Synthesis Netlist Verification

Given the surprising result, the synthesised netlist was verified rather than trusted.

A tooling obstacle. Yosys ships behavioural models at `/usr/share/yosys/xilinx/cells_sim.v`, but this file describes *every* Xilinx primitive including block RAM and DSP slices, and contains constructs Icarus cannot parse. Attempting to include it produced several hundred syntax errors.

Resolution. Since the netlist uses only three primitive types, minimal behavioural models were written for exactly those.

```

1 module LUT4(output O, input I0, I1, I2, I3);
2     parameter [15:0] INIT = 16'h0000;
3     assign O = INIT[{I3, I2, I1, I0}];
4 endmodule
5
6 module IBUF(output O, input I);
7     assign O = I;
8 endmodule
9
10 module OBUF(output O, input I);
11     assign O = I;
12 endmodule

```

Listing 23: Minimal Xilinx primitive models.

The LUT4 model captures exactly how a real LUT operates. `INIT` is a 16-bit constant holding the complete truth table. The four inputs are concatenated with `{I3, I2, I1, I0}` to form a 4-bit index, selecting the corresponding bit of `INIT`. This is not a simplification — it is precisely the mechanism by which a physical LUT implements an arbitrary boolean function.

```

PASS row 0 -> 0
PASS row 1 -> 0
...
PASS row 13 -> 1
ALL 14 PASSED ON SYNTHESIZED NETLIST

```

Listing 24: Post-synthesis verification result.

Significance. This is a stronger claim than the earlier testbench result. It verifies not merely that the *source description* behaves correctly, but that the **actual gate-level circuit produced by synthesis** — the thing that would be programmed onto silicon — is functionally equivalent to the original C++ reference implementation. The full verification chain is:

$$\text{C++ predict()} \equiv \text{SystemVerilog RTL} \equiv \text{Synthesised Xilinx netlist} \quad (13)$$

with each equivalence independently demonstrated across all fourteen examples.

9 Engineering Challenges Encountered

This section documents the practical obstacles met during the project, because the debugging process was a substantial portion of the work and the diagnostic reasoning is more transferable than the specific fixes.

Table 33: Summary of engineering challenges and their resolutions.

Challenge	Diagnosis	Resolution
Toolchain version mismatch	Windows Icarus build was 10.1.1; <code>always_comb</code> needs ≥ 11	Adopt WSL, where <code>apt</code> provides 12.0
WSL install failure (0x80370114)	Required Windows optional features not enabled	Enable WSL and Virtual Machine Platform; reboot
Icarus array-literal limitation	Reduced to a minimal three-line case, isolating the failure to <code>'{...}'</code> syntax itself	Restructure encoding to packed vectors with <code>+</code> : slicing
Truncated file causing misattributed errors	<code>tail -5</code> revealed a file ending mid-statement, missing its <code>endmodule</code>	Rewrite via shell heredoc; verify termination with <code>tail -3</code>
Nonblocking assignment read timing	Isolated in a four-line register test, confirming the testbench was at fault	Insert <code>#1</code> after each <code>@(posedge clk)</code>
Pipeline-exporter divergence	Working <code>.vh</code> was handwritten; exporter still emitted the old format	Rewrite <code>export_vh</code> to emit packed format with computed widths

9.1 Truncated File Producing Misattributed Errors

This was the most instructive debugging episode of the project and merits detailed treatment.

Problem. Compiling three files produced hundreds of syntax errors, all attributed to `minimal_xilinx_prims.v` — a twelve-line file whose contents had been verified byte-for-byte as correct.

Diagnosis. The reported file was verified through escalating checks: visual inspection; `cat -A` (ruling out Windows line endings); and `head -c 50 | xxd` (ruling out a byte-order mark). All confirmed the file was fine.

Attention therefore shifted to the *other* files in the compilation. `tail -5 tb_synth_check.sv` revealed the actual fault — the file ended **mid-statement**, truncated, missing its final test row, its loop, and its closing `endmodule`. It had been corrupted during an earlier editing session in which a command was accidentally typed into `nano` rather than the shell.

Because the truncated file lacked a closing `endmodule`, Icarus’s parser never returned to a clean state and continued reading directly into the *next* file, attributing its confusion to whichever file happened to follow.

Transferable lesson. When a compiler blames file *B* and file *B* is provably correct, examine file *A*. A file that fails to terminate cleanly can propagate parser confusion into everything compiled after it.

9.2 Pipeline–Exporter Divergence

Problem. After switching the header format to packed vectors, the working `.vh` file had been produced by hand rather than generated by the C++ exporter, which still emitted the old unsupported format. The pipeline was silently broken end-to-end: the hardware worked, but only for a hand-transcribed tree.

Significance. This was arguably the most serious issue of the project, precisely because nothing failed visibly. Simulation passed. Synthesis passed. Every test reported success. But the automated path from *any dataset* to *working hardware* had been severed, leaving a system that worked for exactly one hardcoded tree.

Resolution. `export_vh` was rewritten to emit the packed format directly, with bit-widths computed from the tree rather than hardcoded. The regenerated file was confirmed genuine by an observable difference from the hand-written version: the exporter computed `THRESHOLD_BITS` from the actual maximum threshold present, producing a different width than had been assumed by hand.

Transferable lesson. Passing tests do not guarantee a working pipeline. A system can be correct at every individual stage while the *connections* between stages have silently broken. Verifying end-to-end — regenerating artefacts from source rather than reusing cached copies — is a distinct check from verifying each stage.

10 Conceptual Clarifications

Several conceptual points required careful working-through. They are recorded because each represents a genuine conceptual boundary, and because articulating them precisely proved more valuable than the code that resulted.

1. **Entropy zero does not imply non-empty.** `entropy(y) == 0` is satisfied both by a genuinely pure set and by an *empty* set, because “all elements agree” is vacuously true when there are no elements. Code treating zero entropy as evidence that data exists — for instance by indexing `y[0]` — is unsound. This single conflation caused Defect 1.
2. **Indices are meaningful only relative to their container.** A node’s `left_child_index` means “position within *this specific* vector”. Copying nodes into a differently-ordered container silently invalidates every such index. This is not detectable by a compiler and produces output that appears structurally plausible.
3. **Stopping is not optional, and purity is not the only reason.** A tree must sometimes stop while a node is still impure — because attributes are exhausted, because too few examples remain, or because no available split improves anything. Recognising this is what motivates `majority_class` as a fallback.
4. **Recursion is a software luxury.** `predict()` calls itself; hardware cannot. The substitute — a fixed-count unrolled loop plus a freeze flag — is not merely a syntactic translation but a different computational model.

5. **Combinational logic has no concept of time.** `always_comb` is not “fast”; it is *untimed*. Introducing a clock does not make the design faster — it makes it *staged*, which is a different property entirely.
6. **Higher training gain does not imply better generalisation.** The multi-way experiment produced substantially higher information gain and a more compact tree, yet *identical* holdout accuracy.
7. **A design being correct now does not make it general.** The hardware module hardcodes bit-widths that happen to match the current tree. A larger tree would silently truncate index values rather than failing loudly.
8. **Tool support is not uniform.** Icarus accepts unpacked array ports; Yosys does not. Icarus rejects `{...}` array literals; other simulators accept them. A design intended to pass through multiple tools must target the **intersection** of their supported subsets.

11 Limitations

Table 34: Known limitations, with severity and proposed remedy.

Limitation	Consequence	Remedy
Hardware bit-widths not parameterised	A larger tree would be silently truncated , corrupting traversal with no error raised	Reference <code>ROW_BITS</code> etc. directly from the header
Dataset size (14 rows)	Accuracy measurable only to $\pm 1/3$; no statistical claims possible	Move to a larger dataset with k-fold cross-validation
Binary splits on categorical data	Imposes spurious ordering; cannot express clean multi-way partitions	Implement native categorical splitting
No post-pruning	Three hand-tuned hyperparameters instead of one principled α	Implement cost-complexity pruning
No timing analysis	Pipelining comparison complete on area, incomplete on speed	Run Vivado with device-specific timing models
Arbitrary tie-breaking	Wind excluded despite tying exactly with Outlook	Define an explicit tie-breaking policy
Single global tree vector	Training a second tree without clearing would interleave nodes	Encapsulate tree state in a class
Fractional thresholds truncated	A threshold of 37.5 would silently become 37	Adopt a fixed-point representation

11.1 On the Parameterisation Gap

The most serious remaining limitation warrants elaboration. The exporter computes `FEATURE_BITS`, `THRESHOLD_BITS`, and `ROW_BITS` correctly for any tree and writes them into the header. **The hardware module ignores them**, using hardcoded literals instead:

```

1 logic [2:0] row [0:NUM_NODES]; // hardcoded 3 bits
2 logic [1:0] cur_feature_idx; // hardcoded 2 bits

```

The fix is straightforward:

```

1 logic [ROW_BITS-1:0] row [0:NUM_NODES];
2 logic [FEATURE_BITS-1:0] cur_feature_idx;
3 logic [THRESHOLD_BITS-1:0] cur_threshold;

```

This was deliberately deferred rather than overlooked: with the project scoped to a single dataset and the remaining stages time-constrained, documenting the gap was judged more valuable than a change that would alter no current results.

12 Future Work

Table 35: Proposed extensions, ordered by priority.

Priority	Extension	Rationale
1	Parameterise the module	Converts the design from correct-for-one-tree to correct-in-general; small and well-understood
2	Full throughput pipeline	Allow a new input each cycle rather than one per three cycles
3	Deploy to physical hardware	Yields genuine timing closure data and end-to-end validation
4	Larger datasets	Enables meaningful hyperparameter tuning via cross-validation
5	Multi-class support	<code>entropy</code> and <code>majority_class</code> are already class-agnostic
6	Post-pruning	Replaces three hand-tuned parameters with one principled α
7	Ensemble methods	Multiple trees evaluate genuinely in parallel in hardware

12.1 Deployment Path

With a board available, the remaining steps are well-defined:

1. Install Vivado (or the relevant vendor toolchain)
2. Create a project targeting the specific device
3. Write a constraints file assigning module ports to physical pins — for example, mapping `features_packed` to slide switches and `result` to an LED
4. Run full implementation (synthesis, placement, routing)
5. Generate a bitstream and program the device
6. Verify by setting switches to a known weather pattern and confirming the LED matches the expected prediction

13 Conclusion

This project traced a decision tree classifier from information-theoretic first principles to a verified, synthesised digital circuit.

Functionally , the work delivers a complete and verified pipeline: a from-scratch C++ ID3-style learner with pre-pruning; a flattening stage converting the recursive tree into a hardware-consumable indexed table; an automated exporter emitting SystemVerilog with computed minimal bit-widths; two SystemVerilog implementations; and a verification chain establishing equivalence from the C++ reference through the RTL description to the post-synthesis gate-level netlist.

Quantitatively , three results stand out, restated in Table 36.

Table 36: Final results summary.

Optimisation	Before	After	Reduction
Bit-width minimisation	174 cells	99 cells	43%
Combinational $\rightarrow$ pipelined	99 cells	46 cells	53%
Xilinx LUT mapping	—	1 LUT4	—
All fourteen dataset examples pass on the synthesised netlist.			

Methodologically , several practices proved repeatedly valuable. Verifying computed quantities against hand calculations before trusting code caught errors early. Reducing failures to minimal test cases distinguished tool limitations from code defects on three separate occasions. Investigating implausible results rather than accepting them confirmed the single-LUT synthesis was genuine. And documenting negative results — the inconclusive hyperparameter sweep, the multi-way experiment that improved gain without improving accuracy — preserved information that a results-only account would have discarded.

The central technical insight concerns the gap between software and hardware representations. A tree in software is defined by *relationships*: pointers, indices, recursive calls. A tree in hardware must be defined by *fixed data*: a table of numbers of known width, in which relationships are expressed as plain integers. Every significant difficulty in this project — the index-remapping defect, the recursion-to-unrolled-loop transformation, the bit-width analysis, the packed-vector encoding — stems from that single distinction. Understanding it is what makes the translation tractable.

A The Final Tree

```
[Humidity < 1?]
  True:
    [Outlook < 1?]
      True: -> Leaf: predict No
      False: -> Leaf: predict Yes
  False:
    [Outlook < 2?]
      True: -> Leaf: predict Yes
      False: -> Leaf: predict Yes
```

Trained with `max_depth = 2`, `min_samples_split = 5`, `purity_threshold = 0.5`, on all fourteen examples.

B Complete Predictions

Table 37: Predictions for all fourteen training examples.

Row	Outlook	Temp	Humidity	Wind	Predicted	Actual	Match
0	Sunny	Hot	High	Weak	No	No	✓
1	Sunny	Hot	High	Strong	No	No	✓
2	Overcast	Hot	High	Weak	Yes	Yes	✓
3	Rain	Mild	High	Weak	Yes	Yes	✓
4	Rain	Cool	Normal	Weak	Yes	Yes	✓
5	Rain	Cool	Normal	Strong	Yes	No	×
6	Overcast	Cool	Normal	Strong	Yes	Yes	✓
7	Sunny	Mild	High	Weak	No	No	✓
8	Sunny	Cool	Normal	Weak	Yes	Yes	✓
9	Rain	Mild	Normal	Weak	Yes	Yes	✓
10	Sunny	Mild	Normal	Strong	Yes	Yes	✓
11	Overcast	Mild	High	Strong	Yes	Yes	✓
12	Overcast	Hot	Normal	Weak	Yes	Yes	✓
13	Rain	Mild	High	Strong	Yes	No	×
12/14 training accuracy.							

Rows 5 and 13 are misclassified — a direct consequence of `max_depth = 2` preventing the tree from separating them, and an intentional outcome of pre-pruning rather than a defect.

C Reproduction Instructions

```
# 1. Train and export
g++ -std=c++17 -o id3 id3.cpp
./id3                                     # produces decision_tree.vh

# 2. Simulate (combinational)
iverilog -g2012 -o sim decision_tree.sv testbench.sv
vvp sim                                  # expect 14 PASS lines

# 3. Simulate (pipelined)
iverilog -g2012 -o simpipe decision_tree_pipelined.sv tb_pipelined.sv
vvp simpipe                              # expect 2 PASS, 3 cycles each

# 4. View waveforms
gtkwave waves.vcd

# 5. Generic synthesis
yosys -s synth.js                          # expect 99 cells
yosys -s synth_oversized.js               # expect 174 cells
yosys -s synth_pipelined.js               # expect 46 cells

# 6. Xilinx synthesis and netlist verification
yosys -s synth_verify.js                  # writes synth_netlist.v
iverilog -g2012 -o simsynth synth_netlist.v tb_synth_check.sv \
    minimal_xilinx_prims.v
vvp simsynth                             # expect ALL 14 PASSED
```

D Repository Contents

Table 38: Files comprising the project.

File	Purpose
id3.cpp	Complete C++ learner, flattener, and exporter
decision_tree.vh	Generated hardware header
decision_tree.sv	Combinational tree-walk module
decision_tree_pipelined.sv	Pipelined tree-walk module
testbench.sv	14-vector self-checking testbench
tb_pipelined.sv	Clocked testbench
tb_synth_check.sv	Post-synthesis netlist testbench
minimal_xilinx_prims.v	Minimal LUT4/IBUF/OBUF models
decision_tree_oversized.sv/.vh	Oversized comparison design
synth*.ys	Yosys synthesis scripts
waves.vcd	Simulation waveform dump

E Glossary

Table 39: Terminology used in this paper.

Term	Definition
Combinational logic	Logic whose outputs depend only on current inputs, with no clock or memory
Sequential logic	Logic whose outputs depend on stored state, updated on clock edges
Register / flip-flop	A one-bit storage element updating only at a clock edge
LUT	Look-Up Table; an FPGA primitive implementing any boolean function of its inputs via a stored truth table
IBUF / OBUF	Input / output buffer; the interface between internal logic and physical pins
Netlist	A gate-level description of a circuit, produced by synthesis
Synthesis	Conversion of a hardware description into a gate-level netlist
RTL	Register Transfer Level; the abstraction at which SystemVerilog designs are typically written
Testbench	A non-synthesisable module that drives a design and checks its outputs
Bitstream	The binary configuration file programmed onto an FPGA
Critical path	The longest propagation delay through a circuit, determining maximum clock frequency
Entropy	Information-theoretic measure of impurity in a label set
Information gain	Reduction in entropy achieved by a proposed split
Pre-pruning	Halting tree growth during construction
Post-pruning	Removing branches after a tree is fully grown
Don't-care	A field position whose value is never read and may hold any value